Registered ABE for Circuits from Evasive Lattice Assumptions

Xinrui Yang[1], Yijian Zhang[2], Ying Gao[1], and Jie Chen[3]

[1] Beihang University, Beijing, China
[2] University of Wollongong, Wollongong NSW, Australia
[3] Wuhan University, Wuhan, China

Abstract. Attribute-based encryption (ABE) enables fine-grained access control but traditionally depends on a central authority to issue decryption keys. Key-policy registered ABE removes this trust assumption by letting users generate their own keys and register public keys with an untrusted curator, who aggregates them into a compact master public key for encryption.
In this paper, we propose a black-box construction of key-policy registered attribute-based encryption from lattice assumptions in the standard model. Technically, our starting point is the registration-based encryption scheme by Döttling et al. (Eurocrypt, 2023). Building on this foundation, we incorporate the public-coin evasive learning with errors (LWE) assumption and the tensor LWE assumption introduced by Wee (Eurocrypt, 2022) to construct a registered ABE scheme that supports arbitrary bounded-depth circuit policies. Compared to prior private-coin approaches, our scheme is based on more intuitive and transparent security assumptions. Furthermore, the entire construction relies solely on standard lattice-based homomorphic evaluation techniques, without relying on other expensive cryptographic primitives. The scheme also enjoys scalability: the sizes of the master public key, helper decryption key and ciphertext grow polylogarithmically with the number of users. Each user's key pair remains succinct, with both the public and secret keys depending solely on the security parameter and the circuit depth.

1 Introduction

Attribute-based encryption (ABE) [SW05, GPSW06] extends traditional public key encryption, ensuring that encrypted data can be securely accessed by designated users and institutions, thereby realizing fine-grained access control over the encrypted data. For example, in key-policy attribute-based encryption (KP-ABE), the access policy is embedded in the decryption key while the ciphertext is associated with a set of attributes. Only a receiver holding a decryption key sk_f that satisfies $f(\mathbf{w}) = 0$ can correctly decrypt a ciphertext $\mathsf{ct}_{\mathbf{w}}$ with attributes $\mathbf{w}$. Compared with standard public key encryption, although ABE supports more powerful functionalities, it requires the introduction of a trusted central authority that issues secret decryption keys to users based on their attributes/policies. The crucial point is that this authority must retain a long-term master secret key; once an attacker compromises this authority, all ciphertexts in the system can be decrypted. This means the master secret key must be meticulously protected throughout the system's lifecycle.

Early research proposed two technical approaches to overcome the key escrow problem in ABE. Threshold encryption [PS08, KG10] is a common method in which the master secret key is divided and distributed among multiple independent authorities to ensure that no single authority holds complete decryption capability. Similarly, multi-authority ABE [LW11, DKW21] schemes allow multiple authorities to collaboratively manage users' decryption keys. However, the aforementioned methods still carry the risk that if a key distribution center is attacked, secret information may be leaked.

Registered attribute-based encryption. Garg et al. [GHMR18] introduced the notion of registration-based encryption (RBE) to address the key escrow problem in identity-based encryption (IBE). To support more complex access policies, Hohenberger et al. [HLWW23] proposed a registered ABE scheme, which can be viewed as an authority-free variant of ABE. In a registered ABE scheme, the central authority is replaced by a key curator, whose operations are completely transparent. Users generate their own public and private key pairs and register their public keys with the key curator along with a decryption policy. The key curator then updates the master public key of the scheme. Similar to ABE, the master public key mpk can be used to encrypt messages associated with any attribute $\mathbf{w}$. Only the registered users whose policy satisfies $f(\mathbf{w}) = 0$ can correctly decrypt the ciphertext using their private key along with a publicly computable helper decryption key hsk that

binds the user's public key to the current mpk. Since mpk is updated every time a new user joins the system, users must periodically refresh their hsk. However, there is an efficiency guarantee: if N users register, then each user only needs to update their decryption key at most $O(\log N)$ times over the system's lifetime. Moreover, both the master public key and the helper decryption key should remain succinct: $|\mathsf{mpk}|, |\mathsf{hsk}| \leq \mathrm{poly}(\lambda, \log N)$.

A challenge: black-box registered ABE for circuit. Existing registered ABE (including RBE) schemes can be divided into two categories: (1) schemes that utilize non-black-box techniques [1], such as those in [GHMR18, GHM+19, GV20, CES21, FWW23, DMQ25] which apply indistinguishability obfuscation [GGH+16] (or witness encryption [VWW22]) to a cryptographic hash function or use a hash garbling scheme [Yao82] to traverse a Merkle tree. However, the use of these non-black-box cryptographic primitives often leads to inefficiencies; (2) black-box schemes, which rely on concrete assumptions in bilinear pairing groups or lattices, thereby making encryption and decryption processes more efficient. Among these, pairing-based constructions include RBE [GKMR23, FKP23], registered ABE [HLWW23, ZZGQ23, AT24, GLWW24, AAH+25] and multi-authority registered ABE [LWW25].

In contrast, progress in the lattice-based setting remains limited. Although a black-box RBE has been recently proposed in [DKL+23], a corresponding black-box construction of registered ABE over lattices, especially for general circuits, is still missing. The only known lattice-based registered ABE [FWW23] supports general policies, but relies on witness encryption with non-black-box use of cryptographic primitives.

Our Results. In this work, we propose a black-box construction of key-policy registered ABE for arbitrary (bounded-depth) circuit policies from lattice assumptions in the standard model. The selective security of the scheme is based on the public-coin evasive LWE assumption and the tensor LWE assumption introduced by Wee [Wee22]. The scheme supports up to N users and has the following features and parameter configurations[2]:

- The common reference string has size $(N + |S|) \cdot \mathrm{poly}(\lambda, d)$.
- Each user's public key and secret key have size $\mathrm{poly}(\lambda, d)$.
- The aggregated master public key has size $|S| \cdot \mathrm{poly}(\lambda, d)$.
- Each user's helper decryption key has size $\mathrm{poly}(\lambda, d)$.
- The ciphertext has size $|S| \cdot \mathrm{poly}(\lambda, d)$.

Here, let F denote the class of access policies, which can be represented by boolean circuits of depth at most d and are computed on the attribute set S. Each user is updated approximately $\mathrm{poly}\log N$ times. Compared with the registered ABE constructions in [HLWW23, FWW23], our black-box registered ABE offers the following advantages:

1. The security of our scheme relies on public-coin evasive learning with errors (LWE) assumption and tensor LWE assumption. Compared with the private-coin evasive LWE assumption relied upon in [FWW23], public-coin evasive LWE yields a more intuitive structure and public parameters, reducing additional hidden information.
2. Our construction depends solely on standard lattice homomorphic evaluation techniques to support circuits of polynomial size. It employs cryptographic primitives in a completely black-box manner without relying on additional expensive cryptographic tools.
3. Each user's public and private keys, as well as the ciphertext, are succinct and do not depend on the total number of users N; instead, their sizes grow only polynomially with $\log N$.
4. Our registered ABE supports user corruption in the standard model without requiring a random oracle.

Table 1 presents a comparison between our scheme and existing lattice-based registered ABE schemes. It can be observed that, compared to [DKL+23], our construction supports more expressive function classes. Although the registration ABE in [FWW23] also supports general circuits as in our work, it relies on non-black-box techniques, such as function-binding hash and witness encryption based on private-coin evasive LWE. Moreover, several recent works [VWW22, BÜW24, BDJ+24, DJM+25] have raised concerns about the soundness of the private-coin evasive LWE assumption.

[1] Black-box techniques rely only on the input/output behavior of a program, while non-black-box techniques make use of the program's internal code or structure.

[2] We omit all $\mathrm{poly}(\log N)$ factors for clarity.

Table 1. Comparison with existing lattice-base RBE schemes. "private-evasive LWE" and "public-evasive LWE" stand for private-coin and public-coin evasive LWE respectively.

Reference	Function	Black-box	Corruption	Assumption	Standard Model
[DKL+23]	ID-based	✓	✓	LWE	✓
[FWW23]	circuit	✗	✗	private-evasive LWE	✓
[FWW23]	circuit	✗	✓	private-evasive LWE	✗
Ours	circuit	✓	✓	public-evasive + tensor LWE	✓

Concurrent Work. In a concurrent and independent work, Champion et al. [CHW25] proposed a key-policy registered ABE supporting general circuits based on falsifiable lattice assumptions. Their scheme supports corruption queries in the random oracle model. One notable advantage of this scheme is that the ciphertext remains compact (independent of the attribute length), which is particularly valuable in applications such as identity-based distributed broadcast encryption. However, a limitation of their approach is that both the crs and the public key pk grow polynomially with the number of users N, which may pose challenges for scalability.

In addition, Zhu et al. [ZZC+25] constructed a key-policy registered ABE supporting general circuits and allowing an arbitrary number of users. Their scheme is proven secure in the standard model as it can handle corruptions without relying random oracles. A notable strength of their work is that the size of the common reference string crs is independent of the number of users N and consists of uniformly random strings. They propose two constructions. The first is based on the LWE assumption but suffers from a key limitation: the ciphertext size grows polynomially with the number of users N, which is impractical in large-scale settings. To overcome this, the second construction introduces an algebraic obfuscator for matrix PRFs [MPV24], enabling ciphertext compression to $\mathrm{poly}(\log N)$. However, this comes at the cost of higher computational complexity and relies on the stronger private-coin evasive LWE assumption.

Our key-policy registered ABE scheme for circuits is based on the public-coin evasive LWE and tensor LWE assumptions, and is proven secure in the standard model. The scheme avoids reliance on additional cryptographic primitives such as obfuscators, which helps reduce computational overhead. Furthermore, both the pk and ct sizes remain succinct. A limitation of our construction lies in the size of the common reference string, which grows with $N \cdot \mathrm{poly}(\lambda, \log N)$; nevertheless, it is not involved in the encryption or decryption procedures. Overall, the scheme maintains security guarantees while achieving favorable efficiency in both computation and communication, making it a practical candidate solution. A comparison of the three schemes in terms of security assumptions and communication overhead is summarized in Table 2.

Table 2. Comparison with concurrent lattice based key-policy registered ABE. "Obf" stands for the algebraic obfuscator for matrix PRF. Throughout the table, we omit the $\mathrm{poly}(\lambda, d)$ and $\mathrm{poly}(\log N)$ factors for clarity.

Ref	Assumption	Standard Model	$\lvert\mathsf{crs}\rvert$	$\lvert\mathsf{pk}\rvert$	$\lvert\mathsf{ct}\rvert$
[CHW25]	ℓ-succinct LWE	✗	$N^2 + \lvert S\rvert^2$	N	1
[ZZC+25]	LWE	✓	$\lvert S\rvert$	1	$N \cdot \lvert S\rvert$
[ZZC+25]	private-evasive LWE + Obf	✓	$\lvert S\rvert$	1	$\lvert S\rvert$
Ours	public-evasive + tensor LWE	✓	$N + \lvert S\rvert$	1	$\lvert S\rvert$

1.1 Technical Overview

In this section, we provide a high-level overview of our main construction. We begin by introducing some notation. We define the gadget matrix $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g}^\top$ as the diagonal concatenation of the vector $\mathbf{g}$ repeated n times, where $\mathbf{I}_n$ is the n-dimensional identity matrix, $\mathbf{g}^\top = [1, 2, \ldots, 2^{\lceil \log q \rceil - 1}]$, and $\otimes$ denotes the tensor (Kronecker) product. Given a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a vector $\mathbf{y} \in \mathbb{Z}_q^n$, and assuming a trapdoor $\mathbf{A}_\tau^{-1}$ for $\mathbf{A}$ is available, we can efficiently sample a vector $\mathbf{x}$ from a discrete Gaussian distribution such that $\mathbf{A}\mathbf{x} = \mathbf{y}$. We denote this as $\mathbf{x} \leftarrow \mathbf{A}^{-1}(\mathbf{y})$. To simplify the description in the overview, we use underlines in place of noise terms.

Before diving into the construction details, we briefly recall the definition of slotted key-policy registered ABE, which will be used throughout the scheme description and security analysis. A slotted Reg-ABE scheme allows each user generate a public-secret key pair $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathrm{KeyGen}(\mathsf{crs}, i, f)$, where crs is a common reference string generated by the Setup algorithm, $i \in [N]$ is the user's index, and f is the specified policy. Upon joining the system, each user registers their public key pk_i with an untrusted key curator (KC). The KC then uses all registered pk to construct a succinct master public key mpk. Given the mpk, attribute set $\mathbf{w}$ and a message μ, a sender can compute the ciphertext as $\mathsf{ct} \leftarrow \mathrm{Encrypt}(\mathsf{mpk}, \mathbf{w}, \mu)$. As for correctness, any registered user whose policy satisfies $f(\mathbf{w}) = 0$ should be able to correctly recover the message $\mu = \mathrm{Decrypt}(\mathsf{sk}_{rcv}, \mathsf{hsk}_{rcv}, \mathsf{ct})$, where hsk_{rcv} is a helper decryption key generated by KC based on the receiver's index rcv. Regarding security, the ciphertext should hide the message when $f(\mathbf{w}) = 1$, assuming $\mathbf{w}$ is fixed before the crs is generated.

Homomorphic Computation. Our construction relies on the algorithms proposed by [BGG+14], which enables homomorphic computation over matrices and matrix encodings. Let $n, q, \ell \in \mathbb{N}$ and $m = n \log q$. Let $\mathbf{C} \in \mathbb{Z}_q^{n \times \ell m}$ be a randomly sampled matrix. For a Boolean circuit $f : \{0,1\}^\ell \to \{0,1\}$ and an input vector $\mathbf{w} \in \{0,1\}^\ell$, there exist two low-norm matrices $\mathbf{H}_{\mathbf{C},f,\mathbf{w}}$ and $\mathbf{H}_f$ such that

$$(\mathbf{C} - \mathbf{w}^\top \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbf{C},f,\mathbf{w}} = \mathbf{C} \cdot \mathbf{H}_f - f(\mathbf{w}) \cdot \mathbf{G}$$

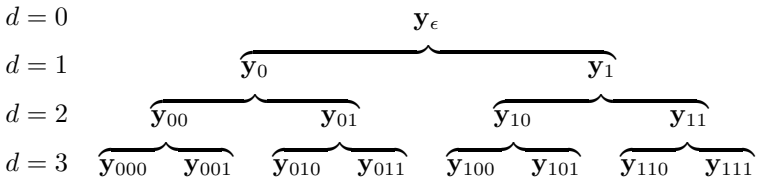
Here, the matrix $\mathbf{H}_f$ depends only on the circuit f, while $\mathbf{H}_{\mathbf{C},f,\mathbf{w}}$ depends on both the circuit f and the input $\mathbf{w}$.

Starting Point: Registration-based Encryption. Our construction of the registered ABE scheme is inspired by the black-box RBE proposed by Döttling et al. [DKL+23]. RBE can be viewed as a special case of registered ABE, instantiated for equality policies defined over identity strings. The main construction idea from [DKL+23] is as follows. By leveraging the algebraic properties of the SIS-based Merkle tree from [LLNW16] in combination with the dual Regev encryption scheme [GPV08], they are able to avoid the use of obfuscation or other non-black-box cryptographic techniques. Specifically, the construction employs an "encryption-friendly" hash function that combines Ajtai's [Ajt96] SIS function with the "binary decomposition" gadget matrix $\mathbf{G}$ from [GPV08], to map elements on the right side of the hash as follows:

$$f(\mathbf{x}_0, \mathbf{x}_1) := \mathbf{A}_0(-\mathbf{G}^{-1}(\mathbf{x}_0)) + \mathbf{A}_1(-\mathbf{G}^{-1}(\mathbf{x}_1)) \mod q,$$

where $\mathbf{A}_0, \mathbf{A}_1 \in \mathbb{Z}_q^{n \times m}$ are uniformly random matrices.

We illustrate how to aggregate user public keys into the master public key mpk using a Merkle tree, and how to perform encryption with mpk, through the following example. Assume the number of users is $N = 8$, and each user generates a public-secret key pair $(\mathsf{pk}_i = \mathbf{y}, \mathsf{sk}_i = \mathbf{x})$, publishing their public key, where $i \in [8]$ denotes the user's index. These 8 users are placed at the leaf nodes of a Merkle tree of depth $k = 3$ and we use $d \in \{0, 1, \ldots, k\}$ to denote the depth of a node within the tree. Specifically, the public key $\mathbf{y}_{ind^{(i)}}$ is assigned as the label of the corresponding leaf node, where the string $ind^{(i)} \in \{0,1\}^k$ is the binary representation of $i - 1$. The labels of the nodes at each level of the Merkle tree are as follows:



Starting from the leaf node, let $\mathbf{y}_{str||0}$ and $\mathbf{y}_{str||1}$ denote the labels of the left and right child nodes of each node $str \in \{0,1\}^{d-1}, d \in [3,1]$, we compute the label of str on the path recursively toward the root ϵ as follows:

$$\mathbf{y}_{str} := \mathbf{A}_0 \cdot (-\mathbf{G}^{-1}(\mathbf{y}_{str||0})) + \mathbf{A}_1 \cdot (-\mathbf{G}^{-1}(\mathbf{y}_{str||1}))$$

Let $\mathbf{u}_{str||0} = -\mathbf{G}^{-1}(\mathbf{y}_{str||0}), \mathbf{u}_{str||1} = -\mathbf{G}^{-1}(\mathbf{y}_{str||0})$. We denote the nodes on the path from the leaf node $ind^{(i)}$ to the root as $ind^{(i)}_{1:3}, ind^{(i)}_{1:2}, ind^{(i)}_{1:1}, ind^{(i)}_{1:0}$, and use $\mathbf{y}_{ind^{(i)}_{1:j}||0}$ and $\mathbf{y}_{ind^{(i)}_{1:j}||1}$ to denote the

labels of the left and right children of node $ind_{1:j}^{(i)}$, respectively[3]. We treat the root label $\mathbf{y}_\epsilon$ as the master public key mpk and the set $\{\mathbf{u}_{ind_{1:j}^{(i)}||0}, \mathbf{u}_{ind_{1:j}^{(i)}||1}\}_{j=0}^{2}$ (i.e. a Merkle tree opening proof) as the helper decryption key $\mathbf{wit}_i$, which is sent to the user with index i. It can be verified that multiplying the index-dependent matrix $\mathbf{T}_i$ with the corresponding helper decryption key yields the following result:

$$\overbrace{\begin{pmatrix} \mathbf{A}_0 & \mathbf{A}_1 & & & & \\ \overline{ind}_1^{(i)} \cdot \mathbf{G} & ind_1^{(i)} \cdot \mathbf{G} & \mathbf{A}_0 & \mathbf{A}_1 & & \\ & & \overline{ind}_2^{(i)} \cdot \mathbf{G} & ind_2^{(i)} \cdot \mathbf{G} & \mathbf{A}_0 & \mathbf{A}_1 \\ & & & & \overline{ind}_3^{(i)} \cdot \mathbf{G} & ind_3^{(i)} \cdot \mathbf{G} \end{pmatrix}}^{\mathbf{T}_i} \cdot \begin{pmatrix} \mathbf{u}_{ind_{1:0}^{(i)}||0} \\ \mathbf{u}_{ind_{1:0}^{(i)}||1} \\ \mathbf{u}_{ind_{1:1}^{(i)}||0} \\ \mathbf{u}_{ind_{1:1}^{(i)}||1} \\ \mathbf{u}_{ind_{1:2}^{(i)}||0} \\ \mathbf{u}_{ind_{1:2}^{(i)}||1} \end{pmatrix} = \begin{pmatrix} \mathbf{y}_\epsilon \\ 0 \\ 0 \\ -\mathbf{y}_{ind}^{(i)} \end{pmatrix}$$

In summary, by introducing the index-dependent matrix $\mathbf{T}_i$ and performing encryption jointly with the master public key $\mathbf{y}_\epsilon$, the 8-slotted RBE scheme is defined as follows:

$$\begin{aligned} \mathsf{crs} &:= \mathbf{B}, \mathbf{A}_0, \mathbf{A}_1 \leftarrow\$ \mathbb{Z}_q^{n\times m} \\ \mathsf{sk}_i &:= \mathbf{x}_i, \qquad \mathsf{pk}_i := \mathbf{y}_i = \mathbf{B} \cdot \mathbf{x}_i \\ \mathsf{mpk} &:= \mathbf{B}, \mathbf{y}_\epsilon, \qquad \mathsf{hsk}_i := \{\mathbf{u}_{ind_{1:j}^{(i)}||0}, \mathbf{u}_{ind_{1:j}^{(i)}||1}\}_{j=0}^{2} \\ \mathsf{ct}_{rcv} &:= \underline{(\mathbf{s}_0^\top, \mathbf{s}_1^\top, \mathbf{s}_2^\top, \mathbf{s}_3^\top) \cdot \mathbf{T}_{rcv}}, \quad \underline{\mathbf{s}_0^\top \cdot \mathbf{y}_\epsilon} + \lfloor q/2 \rfloor \cdot \mu, \quad \underline{\mathbf{s}_3^\top \cdot \mathbf{B}} \end{aligned}$$

where $\mathbf{x}_{ind^{(i)}} \leftarrow \{0,1\}^m$ and $\mathbf{s}_i \leftarrow \mathbb{Z}_q^n$ for $i \in [0,3]$ are randomly sampled vectors, rcv is the specified index. It can be observed that multiplying ciphertext component $\underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_3^\top) \cdot \mathbf{T}_{rcv}}$ with the helper decryption key $\mathbf{wit}_{rcv}$ yields the quantity $\mathbf{s}_0^\top \cdot \mathbf{y}_\epsilon - \mathbf{s}_3^\top \cdot \mathbf{y}_{ind^{(rcv)}}$. Therefore, only the receiver with secret key $\mathbf{x}_{ind^{(rcv)}}$ and the corresponding helper decryption key can decrypt the ciphertext correctly. Decryption computes the following:

$$\underline{\mathbf{s}_0^\top \cdot \mathbf{y}_\epsilon} + \lfloor q/2 \rfloor \cdot \mu - \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_3^\top) \cdot \mathbf{T}_{rcv}} \cdot \mathbf{wit}_{rcv} - \underline{\mathbf{s}_3^\top \cdot \mathbf{B}} \cdot \mathbf{x}_{ind^{(rcv)}} \approx \lfloor q/2 \rfloor \cdot \mu.$$

Implement "One-to-One" Slotted Registered ABE. To construct a key-policy slotted registered ABE scheme, we embed a user-defined access policy f into each user's public key, and embed an attribute vector $\mathbf{w}$ into the ciphertext. Only users whose registered policy satisfies $f(\mathbf{w}) = 0$ can correctly decrypt the ciphertext using their secret key and helper decryption key. Our main idea is to modify the dual Regev encryption scheme [GPV08] while ensuring compatibility with the structure of the SIS-based Merkle tree.

Recall the dual Regev encryption scheme, where the public key is an SIS instance $\mathbf{y} = \mathbf{B} \cdot \mathbf{x}$, and the corresponding secret key is a solution $\mathbf{x} \in \{0,1\}^m$ to this SIS instance. Here, $\mathbf{B} \leftarrow \mathbb{Z}_q^{n\times m}$ is a public random matrix. Building on this, we draw on techniques from the key-policy ABE scheme proposed by Boneh et al. [BGG+14]. First, we perform a homomorphic computation on a public matrix $\mathbf{C} \in \mathbb{Z}_q^{n\times ml}$ to obtain $\mathbf{C}_f \leftarrow \mathbf{C} \cdot \mathbf{H}_f$. Since the distribution of $\mathbf{C}_f$ is indistinguishable from uniform, the resulting public key $\mathbf{y} = (\mathbf{B} \mid \mathbf{C}_f) \cdot \mathbf{x}$ remains statistically indistinguishable from uniform. To encrypt a message μ under the attribute $\mathbf{w}$, the "one-to-one" slotted registered ABE is as follows:

$$\begin{aligned} \mathsf{crs} &:= \mathbf{B}, \mathbf{A}_0, \mathbf{A}_1 \leftarrow\$ \mathbb{Z}_q^{n\times m}, \ \mathbf{C} \leftarrow\$ \mathbb{Z}_q^{n\times m\ell} \\ \mathsf{sk}_i &:= \mathbf{x}_i, \qquad \mathsf{pk}_i := \mathbf{y}_i = (\mathbf{B} \mid \mathbf{C}_f) \cdot \mathbf{x}_i \\ \mathsf{mpk} &:= \mathbf{B}, \mathbf{C}, \mathbf{y}_\epsilon, \qquad \mathsf{hsk}_i := \{\mathbf{u}_{ind_{1:j}^{(i)}||0}, \mathbf{u}_{ind_{1:j}^{(i)}||1}\}_{j=0}^{k-1} \\ \mathsf{ct} &:= \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{T}_{rcv}}, \quad \underline{\mathbf{s}_0^\top \cdot \mathbf{y}_\epsilon} + \lfloor q/2 \rfloor \cdot \mu, \quad \underline{\mathbf{s}_k^\top \cdot \mathbf{B}}, \quad \underline{\mathbf{s}_k^\top \cdot (\mathbf{C} - \mathbf{w} \otimes \mathbf{G})} \end{aligned}$$

To enable decryption using the secret key $\mathbf{x}$, we perform a homomorphic evaluation on the attribute component of the ciphertext, yielding:

$$\underline{\mathbf{s}^\top \cdot (\mathbf{C} - \mathbf{w} \otimes \mathbf{G})} \cdot \mathbf{H}_{\mathbf{C},f,\mathbf{w}} = \underline{\mathbf{s}^\top \cdot (\mathbf{C}_f - f(\mathbf{w}) \cdot \mathbf{G})}.$$

[3] For example, for the leaf node 001, $\mathbf{y}_{00}$ corresponds to $\mathbf{y}_{001_{1:2}}$, and $\mathbf{y}_{000}$ and $\mathbf{y}_{001}$ correspond to $\mathbf{y}_{001_{1:2}||0}$ and $\mathbf{y}_{001_{1:2}||1}$, respectively.

Therefore, when $f(\mathbf{w}) = 0$, decryption proceeds as follows:

$$\lfloor q/2 \rfloor \cdot \mu \approx \underline{\mathbf{s}_0^\top \cdot \mathbf{y}_\epsilon + \lfloor q/2 \rfloor \cdot \mu} - \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{T}_{rcv}} \cdot \mathbf{wit}_{rcv} - \underline{\mathbf{s}_k^\top \cdot (\mathbf{B} \mid \mathbf{C}_f)} \cdot \mathbf{x}_{ind^{(rcv)}}.$$

It follows that correct decryption is only possible for the registered user with index rcv whose policy satisfies $f(\mathbf{w}) = 0$.

Towards "One-to-Many" Constructions. While the above construction ensures correctness, specifying an index in the encryption algorithm limits decryption to a single designated user. To overcome this limitation and enable one-to-many access control, we adopt the evasive LWE assumption recently proposed by Wee [Wee22], and leverage it to construct a registered ABE scheme that supports general circuits. In what follows, we introduce this assumption along with the corresponding modified construction.

Evasive LWE. The public-coin evasive LWE assumption is a strengthened variant of the standard LWE assumption, defined in a setting where the sampler Samp reveals all of its internal randomness to the distinguisher. This characteristic distinguishes it from the private-coin variants, where parts of the sampling process are hidden from the distinguisher.

The core idea is that even when an adversary is given additional information—specifically, a low-norm matrix $\mathbf{K}$ satisfying $\mathbf{BK} = \mathbf{P}$, where $\mathbf{P} \in \mathbb{Z}_q^{n \times t}$ is sampled from a PPT sampler Samp, then the adversary can exploit this information only via the limited means of computing the product $(\mathbf{s}^\top \mathbf{B} + \mathbf{e}) \cdot \mathbf{B}^{-1}(\mathbf{P}) \approx \mathbf{s}^\top \mathbf{P}$ and trying to distinguish this from uniform. More precisely, under the public-coin evasive LWE assumption, there exists a set of conditions:

$$\text{if} \quad (\mathbf{B}, \mathbf{P}, \boxed{\mathbf{s}^\top \mathbf{B} + \mathbf{e}}, \boxed{\mathbf{s}^\top \mathbf{P} + \mathbf{e}''}, \mathsf{aux}) \approx_c (\mathbf{B}, \mathbf{P}, \boxed{\mathbf{c}}, \boxed{\mathbf{c}''}, \mathsf{aux})$$

$$\text{then} \quad (\mathbf{B}, \mathbf{P}, \boxed{\mathbf{s}^\top \mathbf{B} + \mathbf{e}}, \mathbf{B}^{-1}(\mathbf{P}), \mathsf{aux}) \approx_c (\mathbf{B}, \mathbf{P}, \boxed{\mathbf{c}}, \mathbf{B}^{-1}(\mathbf{P}), \mathsf{aux})$$

Here, $\mathbf{B} \leftarrow \mathbb{Z}_q^{n \times m}$, $\mathbf{s} \leftarrow \mathbb{Z}_q^n$ are randomly sampled, $\mathbf{e} \leftarrow \chi^m$, $\mathbf{e}'' \leftarrow \chi^t$ are fresh noise vectors, and aux includes all randomness involved in sampling $\mathbf{P}$. It follows that if $\mathbf{s}^\top \mathbf{P} + \mathbf{e}'' \approx_c \mathbf{c}''$, then the high-order bits of $(\mathbf{s}^\top \mathbf{B} + \mathbf{e}) \cdot \mathbf{B}^{-1}(\mathbf{P}) \approx \mathbf{s}^\top \mathbf{P}$ are pseudorandom, and indistinguishable from uniform by any efficient adversary.

Combining the above evasive LWE assumption, we aim to design an N-slotted registered ABE scheme for a priori bounded number of users N, as follows:

$$\begin{aligned}
\mathsf{crs} &:= \mathbf{B}, \mathbf{A}_0, \mathbf{A}_1, \mathbf{C}, \mathbf{D}, \{\mathbf{D}^{-1}(\mathbf{T}_i)\}_{i \in [N]} \\
\mathsf{sk}_i &:= \mathbf{x}_i, \qquad \mathsf{pk}_i := \mathbf{y}_i = (\mathbf{B} \mid \mathbf{C}_f) \cdot \mathbf{x}_i \\
\mathsf{mpk} &:= \mathbf{B}, \mathbf{C}, \mathbf{D}, \mathbf{y}_\epsilon, \qquad \mathsf{hsk}_i := \{\mathbf{u}_{ind_{1:j}^{(i)}||0}, \mathbf{u}_{ind_{1:j}^{(i)}||1}\}_{j=0}^{k-1}, \ \mathbf{D}^{-1}(\mathbf{T}_i) \\
\mathsf{ct} &:= \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D}}, \quad \underline{\mathbf{s}_0^\top \cdot \mathbf{y}_\epsilon} + \lfloor q/2 \rfloor \cdot \mu, \quad \underline{\mathbf{s}_k^\top \cdot \mathbf{B}}, \quad \underline{\mathbf{s}_k^\top \cdot (\mathbf{C} - \mathbf{w} \otimes \mathbf{G})}
\end{aligned}$$

In the setup algorithm, we compute the matrix $\{\mathbf{T}_i\}_{i \in [N]}$ corresponding to each user index, and use the trapdoor $\mathbf{D}_\tau^{-1}$ to compute the corresponding low-norm matrix $\{\mathbf{L}_i \leftarrow \mathbf{D}^{-1}(\mathbf{T}_i)\}_{i \in [N]}$. The pair $(\mathbf{L}_i, \mathbf{wit}_i)$ is then sent to the registered user with index i as their helper decryption key. In the encryption algorithm, the product $\underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D}}$ replaces the previous multiplication between LWE secret and the index-dependent matrix $\mathbf{T}_{rcv}$. The receiver can use their hsk_{rcv} to compute:

$$\underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D}} \cdot \mathbf{D}^{-1}(\mathbf{T}_{rcv}) \cdot \mathbf{wit}_{rcv} \approx (\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{T}_{rcv} \cdot \mathbf{wit}_{rcv}.$$

When $f(\mathbf{w}) = 0$, decryption relies on:

$$\lfloor q/2 \rfloor \cdot \mu \approx \underline{\mathbf{s}_0^\top \cdot \mathbf{y}_\epsilon} + \lfloor q/2 \rfloor \cdot \mu - \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{T}_{rcv}} \cdot \mathbf{wit}_{rcv} - (\underline{\mathbf{s}_k^\top \cdot \mathbf{B}} \mid \underline{\mathbf{s}_k^\top \cdot (\mathbf{C}_f - f(\mathbf{w}) \cdot \mathbf{G})}) \cdot \mathbf{x}_{ind^{(rcv)}}.$$

Security Flaw: Pseudorandomness Breakdown. Intuitively, evasive LWE assumption implies that we can replace the terms $\underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D}}, \mathbf{D}^{-1}(\mathbf{T}_{rcv})$ with their product $(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{T}_{rcv}$. Therefore, it suffices to show that the message μ remains hidden given the following:

$$\mathbf{D}, \ \mathbf{A}_0, \ \mathbf{A}_1, \ \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{T}_{rcv}}, \ \underline{\mathbf{s}_0^\top \cdot \mathbf{y}_\epsilon} + \lfloor q/2 \rfloor \cdot \mu, \ \underline{\mathbf{s}_k^\top \cdot \mathbf{B}}, \ \underline{\mathbf{s}_k^\top \cdot (\mathbf{C} - \mathbf{w} \otimes \mathbf{G})}$$

The above randomness can be directly simulated from the LWE distribution. However, it is important to note the following: when $rcv = 0$, the adversary can compute $(\underline{\mathbf{s}_0^\top \cdot \mathbf{A}_0 + \mathbf{s}_1^\top \cdot \mathbf{G}},\ \underline{\mathbf{s}_0^\top \cdot \mathbf{A}_1})$; and when $rcv = 1$, the adversary can compute $(\underline{\mathbf{s}_0^\top \cdot \mathbf{A}_0},\ \underline{\mathbf{s}_0^\top \cdot \mathbf{A}_1 + \mathbf{s}_1^\top \cdot \mathbf{G}})$. Subtracting the two reveals $\underline{\mathbf{s}_1^\top \cdot \mathbf{G}}$, from which $\mathbf{s}_1$ can be recovered, thereby breaking ciphertext pseudorandomness and enabling the adversary to recover the message.

N-slotted Registered ABE. To address the aforementioned security limitation, we introduce randomization by computing the tensor product between $\mathbf{T}_i$ and random Gaussian vectors $\mathbf{r}_i \leftarrow \chi^m$. Specifically, we replace each $\mathbf{T}_i$ in the common reference string with $\mathbf{T}_i \otimes \mathbf{r}_i$. Additionally, we integrate an NIZK proof into the KeyGen algorithm to demonstrate knowledge of the secret key. The accordingly ciphertext components are as follows:

$$
\begin{aligned}
\mathsf{crs} &:= \mathbf{B}, \mathbf{A}_0, \mathbf{A}_1, \mathbf{C}, \mathbf{D}, \{\mathbf{D}^{-1}(\mathbf{T}_i \otimes \mathbf{r}_i)\}_{i\in[N]}, \{\mathbf{r}_i, \mathbf{x}'_i\}_{i\in[N]}, \mathsf{crs}_{\mathsf{NIZK}} \\
\mathsf{sk}_i &:= \mathbf{x}_i = \mathbf{x}'_i + \mathbf{x}''_i, \qquad \mathsf{pk}_i := \mathbf{y}_i = (\mathbf{B} \mid \mathbf{C}_f) \cdot \mathbf{x}_i,\ \pi_i \\
\mathsf{mpk} &:= \mathbf{B}, \mathbf{C}, \mathbf{D}, \mathbf{y}_\epsilon \\
\mathsf{hsk}_i &:= \{\mathbf{u}_{ind^{(i)}_{1:j}||0}, \mathbf{u}_{ind^{(i)}_{1:j}||1}\}_{j=0}^{k-1},\ \mathbf{D}^{-1}(\mathbf{T}_i \otimes \mathbf{r}_i),\ \mathbf{r}_i \\
\mathsf{ct} &:= \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D}},\ \underline{\mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m)} + \mu \cdot \mathbf{g},\ \underline{\mathbf{s}_k^\top \cdot (\mathbf{B} \otimes \mathbf{I}_m)},\ \underline{\mathbf{s}_k^\top \cdot ((\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \otimes \mathbf{I}_m)}
\end{aligned}
$$

where $\mathbf{x}'_i, \mathbf{x}''_i \leftarrow \{0,1\}^{2m}$ for $i \in [N]$, $\pi_i \leftarrow \mathsf{NIZK.Prove}\left(\mathsf{crs}_{\mathsf{NIZK}}, \mathcal{C}_{\mathcal{R}}, (\mathbf{B}, \mathbf{C}_f, \mathbf{x}'_i, \mathbf{y}_i), \mathbf{x}''_i\right)$, and $\mathbf{s}_j \leftarrow \mathbb{Z}_q^{nm}$ for $j \in [0,k]$. Before aggregation, we ensure each public key carries a valid proof of knowledge through $\mathsf{IsValid}(\mathsf{crs}, i, f, \mathsf{pk}_i)$. In terms of correctness, combined with the previous analysis, given an index-related matrix $\mathbf{T}_i \in \mathbb{Z}_q^{(k+1)n\times 2km}$, we obtain the vector $(\mathbf{y}_\epsilon, 0, \ldots, 0, -\mathbf{y}_{ind^{(i)}})$ by multiplying $\mathbf{T}_i$ on the right with a low-norm vector $\mathbf{wit}_i$. This property is preserved under tensoring with a random Gaussian vector $\mathbf{r}_i \leftarrow \chi^m$: we can compute $(\mathbf{T}_i \cdot \mathbf{wit}_i) \otimes \mathbf{r}_i$ by right-multiplying with $\mathbf{wit}_i \otimes \mathbf{I}$. Note that this effect cannot be achieved if we replace the tensor product with regular vector multiplication (on the right). Decryption computes the following quantities:

$$
\begin{aligned}
\underline{(\mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m)} + \mu \cdot \mathbf{g}) \cdot \mathbf{r}_{rcv} &\approx \mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} + \mu \cdot \mathbf{g} \cdot \mathbf{r}_{rcv} \\
\underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D}} \cdot \mathbf{D}^{-1}(\mathbf{T}_{rcv} \otimes \mathbf{r}_{rcv}) \cdot \mathbf{wit}_{rcv} &\approx \mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} - \mathbf{s}_k^\top \cdot (\mathbf{y}_{ind^{(rcv)}} \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} \\
\left(\underline{\mathbf{s}_k^\top \cdot (\mathbf{B} \otimes \mathbf{I}_m)} \mid \underline{\mathbf{s}_k^\top \cdot ((\mathbf{C}_f - f(\mathbf{w})\mathbf{G}) \otimes \mathbf{I}_m)}\right) \cdot (\mathbf{x}_{ind^{(rcv)}} \otimes \mathbf{r}_{rcv}) &\approx \mathbf{s}_k^\top \cdot (\mathbf{y}_{ind^{(rcv)}} \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv}
\end{aligned}
$$

When $f(\mathbf{w}) = 0$, the receiver can correctly recover the message μ.

Proof Strategy. It can be observed that the previous adversary is now only able to compute

$$
\left(\underline{\mathbf{s}_0^\top (\mathbf{A}_0 \otimes \mathbf{r}_1) + \mathbf{s}_1^\top (\mathbf{G} \otimes \mathbf{r}_1)},\ \underline{\mathbf{s}_0^\top (\mathbf{A}_1 \otimes \mathbf{r}_1)}\right) \quad \text{and} \quad \left(\underline{\mathbf{s}_0^\top (\mathbf{A}_0 \otimes \mathbf{r}_2)},\ \underline{\mathbf{s}_0^\top (\mathbf{A}_1 \otimes \mathbf{r}_2) + \mathbf{s}_1^\top (\mathbf{G} \otimes \mathbf{r}_2)}\right).
$$

Compared to standard ABE, the proof strategy for slotted registered ABE differs slightly and needs to address two cases: (i) the adversary knows the secret key associated with slot i, and the challenge attributes do not satisfy the policy associated with slot i, i.e., $f(\mathbf{w}) = 1$; (ii) the adversary does not know the key associated with slot i, and the challenge attributes satisfy the corresponding policy, i.e., $f(\mathbf{w}) = 0$. Our security proof proceeds in two steps. First, we rely on the evasive LWE assumption to reduce the scheme's security to a simpler statement that does not involve short Gaussian distributions. Then, we prove this statement under the LWE and tensor LWE assumptions, separately considering the cases where $f(\mathbf{w}) = 1$ and $f(\mathbf{w}) = 0$.

Step 1. We first eliminate the Gaussian preimage associated with $\mathbf{D}$, thereby reducing the security of proving the pseudorandomness of the following terms:

$$
\begin{aligned}
&\mathbf{c}^\top := \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D}}, && \mathbf{c}_k^\top := \underline{\mathbf{s}_k^\top \cdot (\mathbf{B} \otimes \mathbf{I}_m)}, \\
&\mathbf{d}^\top := \underline{\mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m)}, && \boldsymbol{\psi}^\top := \underline{\mathbf{s}_k^\top \cdot ((\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \otimes \mathbf{I}_m)}, \\
&\left\{\tilde{\mathbf{c}}^{(i)\top} := \underline{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot (\mathbf{T}_i \otimes \mathbf{r}_i)},\ \mathbf{r}_i^\top\right\}_{i\in[N]} \\
&\left\{c'^{(i)} := \underline{\mathbf{s}_0^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_\epsilon - \mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_{ind^{(i)}}},\ \mathbf{r}_i^\top\right\}_{i\in[N]}
\end{aligned}
$$

Here, $c'^{(i)} = \tilde{\mathbf{c}}^{(i)\top} \cdot \mathbf{wit}_i$, and $\tilde{\mathbf{c}}^{(i)\top}$ can be decomposed as $(\tilde{\mathbf{c}}_0^{(i)\top}, \ldots, \tilde{\mathbf{c}}_{k-1}^{(i)\top})$, for each $j \in [0, k-1]$:

$$\begin{aligned}\tilde{\mathbf{c}}_j^{(i)\top} &= \mathbf{s}_j^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top)(\mathbf{A}_0 \mid \mathbf{A}_1) + \mathbf{s}_{j+1}^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top)(\overline{ind}_{j+1}^{(i)} \cdot \mathbf{G} \mid ind_{j+1}^{(i)} \cdot \mathbf{G}) + \mathsf{noise} \\ &= \underbrace{\left(\mathbf{s}_j^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top)(\mathbf{A}_0 \mid \mathbf{A}_1) + \mathsf{noise}\right)}_{:=\tilde{\mathbf{c}}_j'^{(i)\top}} + \mathbf{s}_{j+1}^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top)(\overline{ind}_{j+1}^{(i)} \cdot \mathbf{G} \mid ind_{j+1}^{(i)} \cdot \mathbf{G})\end{aligned}$$

Then, we show that replacing each real NIZK proof generated in KeyGen with a simulated proof via the standard zero-knowledge simulator does not change the adversary's view. This reduction relies on the NIZK zero-knowledge property.

Step 2. We prove the pseudorandomness of the above ciphertext components through the following three steps:

- First, we present the expression of $c'^{(i)}$ in the following two cases:
  - When $f(\mathbf{w}) = 1$, the secret key sk is leaked to the adversary. In this case, according to the equation $(\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbf{C},f_i,\mathbf{w}} = \mathbf{C}_{f_i} - f_i(\mathbf{w})\mathbf{G}$ and $f_i(\mathbf{w}) = 1$, we obtain:

    $$c'^{(i)} \approx \mathbf{d}^\top \cdot \mathbf{r}_i - \left(\mathbf{c}_k^\top \mid \boldsymbol{\psi}^\top \cdot \mathbf{H}_{\mathbf{C},f_i,\mathbf{w}}\right) \cdot (\mathbf{x}_{ind^{(i)}} \otimes \mathbf{r}_i^\top) - \underline{\mathbf{s}_k^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{0} \mid \mathbf{G})\mathbf{x}_{ind^{(i)}}}$$

    The soundness of NIZK guarantees $\mathbf{x}_{ind^{(i)}} = \mathbf{x}'_{ind^{(i)}} + \mathbf{x}''_{ind^{(i)}}$ truly underlies the $\mathbf{y}_{ind^{(i)}}$.
  - When $f(\mathbf{w}) = 0$, the public key pk is honestly generated, and the corresponding secret key sk is secret from the adversary. In this case,

    $$c'^{(i)} \approx \mathbf{d}^\top \cdot \mathbf{r}_i - \underline{\mathbf{s}_k^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_{ind^{(i)}}}$$

- Based on the LWE assumption and the leftover hash lemma, we can replace $\mathbf{c}^\top, \mathbf{c}_k^\top, \mathbf{d}^\top$ and $\boldsymbol{\psi}^\top$ with uniformly random values.
- Finally, we prove the pseudorandomness of the remaining components $\tilde{\mathbf{c}}^{(i)\top}$ and $c'^{(i)}$ in the following two cases:
  - When $f(\mathbf{w}) = 1$, for the pseudorandomness of $\tilde{\mathbf{c}}^{(i)\top}$, we reduce it to proving the pseudorandomness of $\{\tilde{\mathbf{c}}_j'^{(i)\top}\}_{j\in[0,k-1]}$. For the pseudorandomness of $c'^{(i)}$, we reduce it to proving the pseudorandomness of $\underline{\mathbf{s}_k^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{0} \mid \mathbf{G})\mathbf{x}'_{ind^{(i)}}}$.

    We prove this using a new assumption, called the tensor LWE assumption, which differs from evasive LWE in that it does not require Gaussian preimages. Intuitively, the tensor operation "amplifies" a single LWE secret $\mathbf{s}$ into N independent secrets $(\mathbf{s}_1, \ldots, \mathbf{s}_N)$. More concretely, suppose an adversary $\mathcal{A}$ is given:

    $$\begin{aligned}&\mathbf{A}_0, \mathbf{A}_1, \{\mathbf{x}'_{ind^{(i)}}, \mathbf{r}_i^\top, \underline{\mathbf{s}_j^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{A}_0 \mid \mathbf{A}_1)}, \underline{\mathbf{s}_k^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{0} \mid \mathbf{G})\mathbf{x}'_{ind^{(i)}}}\}_{i\in[N],j\in[0,k-1]} \\ \approx_c\ &\mathbf{A}_0, \mathbf{A}_1, \{\mathbf{x}'_{ind^{(i)}}, \mathbf{r}_i^\top\}_{i\in[N]}, \{\text{random}, \text{random}\}_{i\in[N],j\in[0,k-1]}\end{aligned}$$

    where $\mathbf{A}_0, \mathbf{A}_1 \leftarrow \mathbb{Z}_q^{n\times m}$ and $\mathbf{x}' \leftarrow \{0,1\}^{2m}$ are sampled uniformly at random. Under the tensor LWE assumption, $\mathcal{A}$'s advantage in distinguishing the two distributions is negligible.
  - When $f(\mathbf{w}) = 0$, the remaining distribution whose pseudorandomness needs to be proven simplifies to:

    $$\mathbf{A}_0, \mathbf{A}_1, \{\mathbf{y}_{ind^{(i)}}, \mathbf{r}_i^\top, \underline{\mathbf{s}_j^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top)(\mathbf{A}_0 \mid \mathbf{A}_1)}, \underline{\mathbf{s}_k^\top(\mathbf{I}_n \otimes \mathbf{r}_i^\top)\mathbf{y}_{ind^{(i)}}}\}_{i\in[N],j\in[0,k-1]}$$

    The pseudorandomness of this distribution is also implied by the tensor LWE assumption.

We refer to Section 3 for the full construction and detailed security proof of our slotted registered ABE scheme. Based on this, a standard registered ABE can be obtained via the power-of-two technique [GHMR18, HLWW23], with the concrete transformation provided in Section 4.

2 Preliminaries

Notation. First, we define the symbols used throughout our work. For any integer $q \geq 2$, let $\mathbb{Z}_q$ represent the ring of integers modulo q, and denotes the integers within $(-q/2, q/2]$. We use bold capital letters (e.g. $\mathbf{A}$) for matrices, bold lowercase letters (e.g. $\mathbf{x}$) for vectors and $[n]$ to represent the set $[1, n]$. We use $ind^{(i)} \in \{0,1\}^k$ be the binary representation of $i-1$, where $i \in [1, N]$ and $k = \log N$, so that $\{ind^{(i)}\}_{i=1}^N$ enumerates all k-bit binary strings in lexicographic order. For a finite set S, we write $x \leftarrow S$ to denote that x is a uniform random draw from S. For a vector $\mathbf{x}$, we let $\|\mathbf{x}\|$ represent its ℓ_2 norm, and $\|\mathbf{x}\|_\infty$ represents its infinity norm. If a function $f(n)$ is $O(n^{-c})$ for all $c > 0$, we say it is *negligible*, denoted by $\mathsf{neg}(n)$. If $f(n)$ is $O(n^c)$ for some $c > 0$, we say it is *polynomial*, denoted by $\mathrm{poly}(n)$.

Tensor Product. In this work, similarly to [Wee22], we use the tensor product techniques. Let $\mathbf{A} = (a_{i,j}) \in \mathbb{Z}_q^{\ell \times m}$ and $\mathbf{B} \in \mathbb{Z}_q^{n \times p}$. The tensor product of $\mathbf{A}$ and $\mathbf{B}$ is defined as:

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{1,1}\mathbf{B} & \cdots & a_{1,m}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{\ell,1}\mathbf{B} & \cdots & a_{\ell,m}\mathbf{B} \end{bmatrix} \in \mathbb{Z}_q^{\ell n \times mp}.$$

The tensor product satisfies the mixed-product property, which states that for any compatible matrices $\mathbf{A} \in \mathbb{Z}_q^{\ell \times m}, \mathbf{B} \in \mathbb{Z}_q^{n \times p}, \mathbf{C} \in \mathbb{Z}_q^{m \times u}, \mathbf{D} \in \mathbb{Z}_q^{p \times v}$,

$$(\mathbf{A} \otimes \mathbf{B})(\mathbf{C} \otimes \mathbf{D}) = (\mathbf{AC}) \otimes (\mathbf{BD}) \in \mathbb{Z}_q^{\ell n \times uv}$$

As a useful corollary of the mixed-product property, for any pair of vectors $\mathbf{u}, \mathbf{v} \in \mathbb{Z}_q^n$, the following identities hold:

$$\mathbf{u} \otimes \mathbf{v} = (\mathbf{u} \otimes \mathbf{I}_n)(\mathbf{1} \otimes \mathbf{v}) = (\mathbf{I}_n \otimes \mathbf{v})(\mathbf{u} \otimes \mathbf{1}) = (\mathbf{u} \otimes \mathbf{I}_n)\mathbf{v} = (\mathbf{I}_n \otimes \mathbf{v})\mathbf{u}.$$

Note that we follow the convention that matrix multiplication takes precedence over the tensor product. For example, $\mathbf{A} \otimes \mathbf{BC} = \mathbf{A} \otimes (\mathbf{BC})$.

2.1 Lattices Background

Discrete Gaussian Distribution. Let $D_{\mathbb{Z}^m,\sigma}$ be the truncated discrete Gaussian distribution over $\mathbb{Z}^m$ with parameter σ, i.e., we replace the output with $\mathbf{0}$ when the $\|\cdot\|_\infty$ norm exceeds $\sqrt{m} \cdot \sigma$ ($D_{\mathbb{Z}^m,\sigma}$ is bounded by $\sqrt{m} \cdot \sigma$).

Lemma 2.1 (Smudging Lemma [WWW22]) *Let λ be a security parameter. Take any $a \in \mathbb{Z}$ such that $|a| \leq B$. Suppose $\chi \geq B\lambda^{\omega(1)}$. Then, the statistical distance between the distributions $\{z : z \leftarrow D_{\mathbb{Z},\chi}\}$ and $\{z + a : z \leftarrow D_{\mathbb{Z},\chi}\}$ is negligible in λ.*

Leftover Hash Lemma. Let $n, m, q \in \mathbb{N}$ with $m \geq 2n \log q$. We have

$$\left\{(\mathbf{A}, \mathbf{Ax}) : \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m},\ \mathbf{x} \leftarrow \{0,1\}^m\right\} \approx_s \left\{(\mathbf{A}, \mathbf{u}) : \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m},\ \mathbf{u} \leftarrow \mathbb{Z}_q^n\right\}$$

Next, we present a variant of the generalized leftover hash lemma based on [ABB10]:

Lemma 2.2 (Ternary Generalized Leftover Hash Lemma) *Assuming $m \geq 2n \log q / \log 3$ and $q \geq 2$ is square free, let $\mathbf{R}$ be a $m \times k$ matrix chosen uniformly at random from $\{0,1,2\}^{m \times k}$, where $k = k(n)$ is a polynomial in n. Matrices $\mathbf{A}$ and $\mathbf{B}$ are chosen uniformly at random from $\mathbb{Z}_q^{n \times m}$ and $\mathbb{Z}_q^{n \times k}$ respectively. For all fixed vectors $\mathbf{w} \in \mathbb{Z}_q^m$, the statistical indistinguishability between the distributions $(\mathbf{A}, \mathbf{AR}, \mathbf{R}^\top \mathbf{w})$ and $(\mathbf{A}, \mathbf{B}, \mathbf{R}^\top \mathbf{w})$ holds.*

Gadget Matrix. Let $n, q \in \mathbb{Z}$, $g = (1, 2, 4, \ldots, 2^{\lceil \log q \rceil - 1}) \in \mathbb{Z}_q^{\lceil \log q \rceil}$, and $m = n\lceil \log q \rceil$. The gadget matrix $\mathbf{G}$ is defined as the n-fold diagonal concatenation of the vector g. Formally, $\mathbf{G} = \mathbf{g} \otimes \mathbf{I} \in \mathbb{Z}_q^{n \times m}$, where $\mathbf{I} \in \mathbb{Z}_q^{n \times n}$ represents the identity matrix, and $\otimes$ denotes the tensor product.

Trapdoor and Preimage Sampling. We recall the algorithm of trapdoor generation and preimage sampling [GPV08, MP12].

Lemma 2.3 (Gadget Trapdoor) *Let the security parameter $n \geq 1$, $q \geq 2$, and $m \geq 2n \log q$. Efficient algorithms* TrapGen *and* SamplePre *are defined as follows:*

- $\mathsf{TrapGen}(1^n, 1^m, q)$: *On input the lattice dimension n, the modulus q, and the number of samples m, the trapdoor generation algorithm outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m}$.*
- $\mathsf{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \tau)$: *On input a full rank matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a short basis $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m}$ of $\Lambda_q^\perp(\mathbf{A})$, a random vector $\mathbf{u} \in \mathbb{Z}_q^n$, and a Gaussian width parameter τ, the preimage sampling algorithm outputs a vector $\mathbf{x}$, satisfying $\mathbf{A} \cdot \mathbf{x} = \mathbf{u} \mod q$.*

Moreover, the above algorithms satisfy the following properties:

- **Trapdoor Distribution:** *If $(\mathbf{A}, \mathbf{T}) \leftarrow \mathsf{TrapGen}(1^n, 1^m, q)$, the distribution of $\mathbf{A}$ is $\mathsf{negl}(n)$-close to uniform, and $\mathbf{T}_\mathbf{A}$ serves as a short basis for the lattice $\Lambda_q^\perp(\mathbf{A})$ which is a full-rank matrix and satisfies $\|\mathbf{T}_\mathbf{A}\| \leq O(m \log q)$.*
- **Preimage Distribution:** *There exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{T}_\mathbf{A} \in \mathbb{Z}_q^{m \times m}$ where $\mathbf{T}_\mathbf{A}$ is a gadget trapdoor for $\mathbf{A}$ (i.e., $\mathbf{A} \cdot \mathbf{T}_\mathbf{A} = \mathbf{G}_n$), for all $\tau \geq m\|\mathbf{T}\| \log n$ and for all target vectors $\mathbf{u} \in \mathbb{Z}_q^n$, the statistical distance between the following distributions is at most $\mathsf{negl}(n)$:*

$$\{\mathbf{x} \leftarrow \mathsf{SamplePre}(\mathbf{A}, \mathbf{T}_\mathbf{A}, \mathbf{u}, \tau)\} \approx_s \{\mathbf{x} \leftarrow \mathcal{D}_{\mathbb{Z}^m, \tau}\}$$

Homomorphic evaluation. The following is an abstraction of the evaluation procedure in previous ABE schemes [BGG+14].

Lemma 2.4 (Fully Homomorphic Computation [BGG+14]) *Let $n = n(\lambda)$, $q = q(\lambda)$ be lattice parameters. Take any $m \geq n\lceil \log q \rceil$, and let $\ell = \ell(\lambda)$ be an input length. Let $\mathcal{F} = \{F_\lambda\}_{\lambda \in \mathbb{N}}$ be a family of functions $f : \{0,1\}^\ell \to \{0,1\}$ that can be computed by a Boolean circuit of depth at most $d = d(\lambda)$. There exists a pair of deterministic algorithms* $(\mathsf{EvalF}, \mathsf{EvalFX})$ *with the following properties:*

- $\mathsf{EvalF}(\mathbf{A}, f) \to \mathbf{H}_f$. *On input a matric $\mathbf{A} \in \mathbb{Z}_q^{n \times \ell m}$ and a function $f : \{0,1\}^\ell \to \{0,1\}$. The algorithm outputs $\mathbf{H}_f \in \mathbb{Z}_q^{\ell m \times m}$.*
- $\mathsf{EvalFX}(\mathbf{A}, f, \mathbf{x}) \to \mathbf{H}_{\mathbf{A}, f, \mathbf{x}}$. *On input a matric $\mathbf{A} \in \mathbb{Z}_q^{n \times \ell m}$, a function $f : \{0,1\}^\ell \to \{0,1\}$ and an input $\mathbf{x} \in \{0,1\}^\ell$. The algorithm outputs $\mathbf{H}_{\mathbf{A}, f, \mathbf{x}} \in \mathbb{Z}_q^{\ell m \times m}$, such that*

$$(\mathbf{A} - \mathbf{x} \otimes \mathbf{G})\mathbf{H}_{\mathbf{A}, f, \mathbf{x}} = \mathbf{A}\mathbf{H}_f - f(\mathbf{x})\mathbf{G} \quad \textit{and} \quad \|\mathbf{H}_{\mathbf{A}, f, \mathbf{x}}\|_\infty \leq m^{O(d)}$$

2.2 Evasive LWE and Tensor LWE

Learning with Errors (LWE). Given $n, m, q, \chi \in \mathbb{N}$. The learning with error (LWE) assumption $\mathsf{LWE}_{n,m,q,\chi}$ holds if for any PPT adversary $\mathcal{A}$:

$$\left|\Pr\left[\mathcal{A}(\mathbf{A}, \mathbf{s}^\top \mathbf{A} + \mathbf{e}^\top) \to 1\right] - \Pr\left[\mathcal{A}(\mathbf{A}, \mathbf{u}^\top) \to 1\right]\right| \leq \mathrm{negl}(\lambda)$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{s} \leftarrow \mathbb{Z}_q^n, \mathbf{e} \leftarrow \mathcal{D}_{\mathbb{Z}^m, \chi}, \mathbf{u} \leftarrow \mathbb{Z}_q^m$. Regev and Peikert [Reg09, Pei09] proved the existence of a B-bounded error distribution, such that solving $\mathsf{LWE}_{n,m,q,\chi}$ and approximating worst-case lattice problems to a factor of $\tilde{O}(n \cdot q/B)$ are equally hard. We say that the $\mathsf{LWE}_{n,m,q,\chi}$ problem is subexponentially hard if there exists a constant $0 < \epsilon < 1$ such that, for all PPT adversaries $\mathcal{A}$, the above probability is bounded by $2^{-n^\epsilon} \cdot \mathsf{negl}(\lambda)$.

Lemma 2.5 (Implications from LWE [ARYY23]) *Let $n, m, N, q, \chi \in \mathbb{N}$ be parameters. If LWE$(n, (m+1)N, q, \chi)$ holds, then, the following distributions are computationally indistinguishable:*

$$\left(\mathbf{B}_0, \mathbf{B}_1, \mathbf{s}^\top(\mathbf{B}_0 \otimes \mathbf{I}_m) + \mathbf{e}_0^\top, \mathbf{s}^\top \mathbf{B}_1 + \mathbf{e}_1^\top\right) \approx_c (\mathbf{B}_0, \mathbf{B}_1, \mathbf{c}_0, \mathbf{c}_1)$$

where $\mathbf{B}_0 \leftarrow \mathbb{Z}_q^{n \times N}$, $\mathbf{B}_1 \leftarrow \mathbb{Z}_q^{nm \times Nm}$, $\mathbf{s} \leftarrow \mathbb{Z}_q^{nm}$, $\mathbf{e}_0 \leftarrow D_{\mathbb{Z}^N, \chi}$, $\mathbf{e}_1 \leftarrow D_{\mathbb{Z}^{Nm}, \chi}$, $\mathbf{c}_0 \leftarrow \mathbb{Z}_q^N$, and $\mathbf{c}_1 \leftarrow \mathbb{Z}_q^{Nm}$.

Evasive LWE. Let $n, m, p, t, q \in \mathbb{N}$ be parameters, and let λ be a security parameter. Let χ be a parameter for Gaussian distributions. Let Samp be a PPT algorithm that, on input 1^λ, outputs

$$\mathbf{A}' \in \mathbb{Z}_q^{n\times p}, \quad \mathbf{P} \in \mathbb{Z}_q^{n\times t}, \quad \mathsf{aux} \in \{0,1\}^*$$

For a PPT adversary, we define the following advantage functions:

$$\begin{aligned}\mathsf{Adv}^{\mathcal{A}_0}_{\mathsf{PRE}}(\lambda) := &\Pr\left[\mathcal{A}_0(\mathbf{A}', \mathbf{B}, \mathbf{s}^\top\mathbf{A}' + \mathbf{e}'^\top, \mathbf{s}^\top\mathbf{B} + \mathbf{e}^\top, \mathbf{s}^\top\mathbf{P} + \mathbf{e}''^\top, \mathsf{aux}) = 1\right] \\ &- \Pr\left[\mathcal{A}_0(\mathbf{A}', \mathbf{B}, \mathbf{c}, \mathbf{c}_0, \mathbf{c}', \mathsf{aux}) = 1\right],\end{aligned}$$

$$\begin{aligned}\mathsf{Adv}^{\mathcal{A}_1}_{\mathsf{POST}}(\lambda) := &\Pr\left[\mathcal{A}_1(\mathbf{A}', \mathbf{B}, \mathbf{s}^\top\mathbf{A}' + \mathbf{e}'^\top, \mathbf{s}^\top\mathbf{B} + \mathbf{e}^\top, \mathbf{K}, \mathsf{aux}) = 1\right] \\ &- \Pr\left[\mathcal{A}_1(\mathbf{A}', \mathbf{B}, \mathbf{c}, \mathbf{c}_0, \mathbf{K}, \mathsf{aux}) = 1\right]\end{aligned}$$

where

$$(\mathbf{A}', \mathbf{P}, \mathsf{aux}) \leftarrow \mathsf{Samp}(1^\lambda), \quad \mathbf{B} \leftarrow \mathbb{Z}_q^{n\times m}, \quad \mathbf{s} \leftarrow \mathbb{Z}_q^n,$$

$$\mathbf{c} \leftarrow \mathbb{Z}_q^p, \quad \mathbf{c}_0 \leftarrow \mathbb{Z}_q^m, \quad \mathbf{c}' \leftarrow \mathbb{Z}_q^t, \quad \mathbf{e} \leftarrow D_{\mathbb{Z}^m,\chi}, \quad \mathbf{e}' \leftarrow D_{\mathbb{Z}^p,\chi},$$

$$\mathbf{e}''_i \leftarrow D_{\mathbb{Z}^t,\chi}, \quad \mathbf{K} \leftarrow \mathbf{B}^{-1}(\mathbf{P}) \text{ with standard deviation } O(\sqrt{m\log q}).$$

We say that the evasive LWE assumption holds if, for every PPT Samp and $\mathcal{A}_1$, there exists another PPT $\mathcal{A}_0$ and a polynomial $\mathrm{poly}(\cdot)$ such that

$$\mathsf{Adv}^{\mathcal{A}_0}_{\mathsf{PRE}}(\lambda) \geq \mathsf{Adv}^{\mathcal{A}_1}_{\mathsf{POST}}(\lambda)/\mathrm{poly}(\lambda) - \mathsf{negl}(\lambda).$$

Remark 2.6 (Sampler restrictions) *We only require the assumption to hold for samplers in which the auxiliary information explicitly includes the full randomness used by the sampling algorithm Samp. This restriction rules out pathological counter-examples based on program obfuscation, where aux may hide an obfuscated trapdoor for the matrix* $\mathbf{P}$*.*

Remark 2.7 (Noise parameters) *For ease of presentation, we stated the assumption under the simplifying condition that all LWE error terms* $\mathbf{e}, \mathbf{e}'$ *share the same Gaussian parameter* χ*. Nevertheless, the assumption and the scheme naturally extend to a relaxed setting where the error terms in the post-condition are sampled with a larger Gaussian parameter than those in the pre-condition.*

Tensor LWE. Let $n, m, q, \ell, Q \in \mathbb{N}$ be parameters and let $\gamma, \chi > 0$ be Gaussian parameters. For all $\mathbf{x}_1, \ldots, \mathbf{x}_Q \in \{0,1\}^\ell$, the following holds:

$$\mathbf{A}, \left\{\mathbf{s}^\top(\mathbf{I}_n \otimes \mathbf{r}_i)(\mathbf{A} - \mathbf{x}_i \otimes \mathbf{G}) + \mathbf{e}_i^\top, \mathbf{r}_i\right\}_{i\in[Q]} \approx_c \mathbf{A}, \{\mathbf{c}_i, \mathbf{r}_i\}_{i\in[Q]}$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n\times \ell m}$, $\mathbf{s} \leftarrow \mathbb{Z}_q^{mn}$, $\mathbf{e}_i \leftarrow D_{\mathbb{Z}^{\ell m},\chi}$, $\mathbf{r}_i \leftarrow D_{\mathbb{Z}^m,\gamma}$, and $\mathbf{c}_i \leftarrow \mathbb{Z}_q^{\ell m}$.

To justify the plausibility of the tensor LWE assumption, we investigate conditions under which it can be reduced to the standard LWE assumption. As a starting point, we recall the following lemmas, which are implicit in [ARYY23].

Lemma 2.8 (Implicitly proved in [ARYY23]) *Let* $n, m, q, \ell, Q, \beta \in \mathbb{N}$ *be parameters, and let* χ_0, χ, γ *be Gaussian parameters satisfying* $m = \Omega(n\log q)$*,* $\gamma = \Omega(\sqrt{n\log q})$*, and* $\chi = \gamma\chi_0\lambda^{\omega(1)}$*. Then, the hardness of the* $\mathsf{LWE}(n, m, q, \chi_0)$ *problem implies that:*

$$\mathbf{A}, \left\{\mathbf{s}^\top(\mathbf{I}_n \otimes \mathbf{r}_i)\mathbf{A} + \mathbf{e}_i^\top, \mathbf{r}_i\right\}_{i\in[Q]} \approx_c \mathbf{A}, \{\mathbf{c}_i, \mathbf{r}_i\}_{i\in[Q]}$$

where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n\times \ell m}$*,* $\mathbf{s} \leftarrow \mathbb{Z}_q^{mn}$*,* $\mathbf{e}_i \leftarrow D_{\mathbb{Z}^{\ell m},\chi}$*,* $\mathbf{r}_i \leftarrow D_{\mathbb{Z}^m,\gamma}$*, and* $\mathbf{c}_i \leftarrow \mathbb{Z}_q^{\ell m}$*.*

We now introduce a new lemma that proves the same implication between LWE and Tensor LWE in another particular case. The formal proof is provided in the Appendix B.

Lemma 2.9 *Let* $n, m, q, Q \in \mathbb{N}$ *be parameters, and let* $\chi_0, \chi, \gamma > 0$ *be Gaussian parameters satisfying* $m = \Omega(n\log q)$*,* $\gamma = \lambda^{\omega(1)}$*, and* $\chi = \gamma\chi_0\lambda^{\omega(1)}$*. Then, the hardness of the* $\mathsf{LWE}(n, m, q, \chi_0)$ *problem implies that:*

$$\left\{\mathbf{s}^\top(\mathbf{I}_n \otimes \mathbf{r}_i)\mathbf{b}_i + e_i^\top, \mathbf{b}_i, \mathbf{r}_i\right\}_{i\in[Q]} \approx_c \{c_i, \mathbf{b}_i, \mathbf{r}_i\}_{i\in[Q]}$$

where $\mathbf{b}_i \in \mathbb{Z}_q^n$*,* $\mathbf{s} \leftarrow \mathbb{Z}_q^{mn}$*,* $e_i \leftarrow D_{\mathbb{Z},\chi}$*,* $\mathbf{r}_i \leftarrow D_{\mathbb{Z}^m,\gamma}$ *and* $\mathbf{c}_i \leftarrow \mathbb{Z}_q$*.*

2.3 Non-Interactive Zero Knowledge

A non-interactive zero knowledge (NIZK) proof system for the NP problem is a tuple of polynomial-time algorithms $\Pi_{\mathsf{NIZK}} = (\mathsf{Setup}, \mathsf{Prove}, \mathsf{Verify})$ specified as follows:

$\mathsf{Setup}(1^\lambda)$: On input the security parameter 1^λ, the randomized algorithm outputs a common reference string crs.

$\mathsf{Prove}(\mathsf{crs}, C, y, w)$: On input the common reference string crs, a Boolean circuit $C : \{0,1\}^n \times \{0,1\}^h \to \{0,1\}$, a statement $y \in \{0,1\}^n$, a witness $w \in \{0,1\}^h$, the prove algorithm outputs a proof π.

$\mathsf{Verify}(\mathsf{crs}, C, y, \pi)$: On input the common reference string crs, a Boolean circuit $C : \{0,1\}^n \times \{0,1\}^h \to \{0,1\}$, a statement $y \in \{0,1\}^n$, and a proof π, the verification algorithm outputs either 0 or 1.

We say that a NIZK for C is correct if for all crs output by $\mathsf{Setup}(1^\lambda)$, and any $C(y,w) = 1$, we have that $\mathsf{Verify}(\mathsf{crs}, C, y, \mathsf{Prove}(\mathsf{crs}, C, y, w)) = 1$. Below, We define two properties of a NIZK proof.

Definition 2.10 (Zero-knowledge) *A NIZK Π for C satisfies zero-knowledge if there exists a PPT simulator $\mathsf{Z} := (\mathsf{Z}_0, \mathsf{Z}_1)$ such that Z_0 outputs crs and a simulation trapdoor ζ and for all PPT distinguishers D, we have*

$$\left|\Pr\left[\mathsf{D}^{\mathsf{Prove}(\mathsf{crs},\cdot)}(\mathsf{crs}) = 1 : \mathsf{crs} \leftarrow\!\!\$\ \mathsf{Setup}\left(1^\lambda\right)\right] - \Pr\left[\mathsf{D}^{\mathsf{O}(\zeta,(\cdot,\cdot))}(\mathsf{crs}) = 1 : (\mathsf{crs}, \zeta) \leftarrow\!\!\$\ \mathsf{Z}_0\left(1^\lambda\right)\right]\right| \leq \mathsf{neg}(\lambda),$$

where the oracle $\mathsf{O}(\zeta, (\cdot,\cdot))$ takes as input (y,w) and returns $\mathsf{Z}_1(\zeta, C, y)$, if $C(y,w) = 1$, and $\perp$ otherwise.

Definition 2.11 (Knowledge soundness) *A NIZK Π for C satisfies knowledge soundness if there exists a PPT extractor $\mathsf{K} := (\mathsf{K}_0, \mathsf{K}_1)$ such that The distribution of crs output by the algorithm K_0 is indistinguishable from the computation of $\mathsf{Setup}(1^\lambda)$, and for all PPT adversaries $\mathcal{A}$, we have*

$$\Pr\left[\mathsf{Verify}(\mathsf{crs}, C, y, \pi) = 1 \wedge C(y,w) = 0 : \begin{array}{c}(\mathsf{crs}, \xi) \leftarrow\!\!\$\ \mathsf{K}_0\left(1^\lambda\right) \\ (y, \pi) \leftarrow\!\!\$\ \mathcal{A}(\mathsf{crs}) \\ w \leftarrow\!\!\$\ \mathsf{K}_1(\xi, C, y, \pi)\end{array}\right] \leq \mathsf{neg}(\lambda).$$

2.4 Slotted Registered Attribute-Based Encryption

We recall the notion of a slotted registered attribute-based encryption (ABE) scheme adapted from [HLWW23], and modify it accordingly to present the definition of key-policy slotted registered ABE. In a slotted registered ABE scheme, there is an a priori bound on the number of users N, and each user is associated with a unique slot index $i \in [N]$. Each user generates their public key with respect to their designated slot index i. The scheme includes an aggregation algorithm that, given a collection of N public keys, aggregates them into a short master public key. Importantly, in a slotted registered ABE scheme, there is no notion of dynamic user registration; the set of users is fixed at the time of system initialization. We give the definition for the slotted primitive here and defer the full definition of registered ABE to Appendix A.

Let λ be a security parameter and $d, \ell, N \in \mathbb{N}$. A key-policy slotted registered attribute-based encryption for circuit with attribute space $\mathcal{X}$ and policy space $\mathcal{P}$ is a tuple of algorithms with the following syntax:

$\mathsf{Setup}(1^\lambda, 1^N, 1^d, 1^\ell) \to \mathsf{crs}$: On input the security parameter λ, the number of slots N, the circuit depth d and the input length ℓ, the setup algorithm outputs a common reference string crs.

$\mathsf{KeyGen}(\mathsf{crs}, i, f) \to (\mathsf{pk}_i, \mathsf{sk}_i)$: On input the crs, a slot index $i \in [N]$, and a policy $f \in \mathcal{P}_{d,\ell}$, the key generation algorithm outputs a public key pk_i and a secret key sk_i for slot i.

$\mathsf{IsValid}(\mathsf{crs}, i, f, \mathsf{pk}_i) \to b$: On input the crs, a slot index $i \in [N]$, a policy $f \in \mathcal{P}_{d,\ell}$, and a public key pk_i, the key-validation algorithm outputs a bit $b \in \{0,1\}$. This algorithm is deterministic.

$\mathsf{Aggregate}(\mathsf{crs}, \mathsf{pk}_1, \ldots, \mathsf{pk}_N) \to (\mathsf{mpk}, \mathsf{hsk}_1, \ldots, \mathsf{hsk}_N)$: On input crs and a list of public keys $\mathsf{pk}_1, \ldots, \mathsf{pk}_N$, the aggregation algorithm outputs the master public key mpk and a collection of helper decryption keys $\mathsf{hsk}_1, \ldots, \mathsf{hsk}_N$. This algorithm is deterministic.

$\mathsf{Encrypt}(\mathsf{mpk}, \mathbf{w}, \mu) \to \mathsf{ct}$: On input the master public key mpk, an attribute $\mathbf{w} \in \mathcal{X}$, and a message $\mu \in \{0,1\}$, the encryption algorithm outputs a ciphertext ct.

$\mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, \mathbf{w}, \mathsf{ct}) \to \mu$: On input a secret key sk, the corresponding helper decryption key hsk, an attribute $\mathbf{w}$, and a ciphertext ct, the decryption algorithm outputs a message $\mu \in \{0,1\}$. This algorithm is deterministic.

Completeness. For all $\lambda, N, d, \ell \in \mathbb{N}$, all $i \in [N]$ and all $f_1, \ldots, f_N \in \mathcal{P}_{d,\ell}$,

$$\Pr\left[\mathsf{IsValid}(\mathsf{crs}, i, f, \mathsf{pk}_i) = 1 : \begin{array}{c} \mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^N, 1^d, 1^\ell) \\ (\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, i, f) \end{array}\right] = 1.$$

Correctness and Compactness. For all $\lambda, N, d, \ell \in \mathbb{N}$, all $i \in [N]$, all $f_1, \ldots, f_N \in \mathcal{P}_{d,\ell}$, all $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^N, 1^d, 1^\ell)$, all $(\mathsf{pk}_i, \mathsf{sk}_i) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, i, f_i)$, all collections of tuples (j, f_j, pk_j) where $\mathsf{IsValid}(\mathsf{crs}, j, f_j, \mathsf{pk}_j) = 1$, all $\mathbf{w} \in \{0,1\}^\ell$ such that $f_i(\mathbf{w}) = 0$, and all $\mu \in \mathcal{M}$, correctness requires that:

$$\Pr\left[\mathsf{Dec}(\mathsf{sk}_i, \mathsf{hsk}_i, \mathbf{w}, \mathsf{Enc}(\mathsf{mpk}, \mathbf{w}, \mu)) = \mu\right] = 1$$

where $(\mathsf{mpk}, \{\mathsf{hsk}_i\}_{i\in[N]}) \leftarrow \mathsf{Aggregate}\left(\mathsf{crs}, \{\mathsf{pk}_i\}_{i\in[N]}\right)$. Moreover, compactness requires that

$$|\mathsf{mpk}| = \mathrm{poly}(\lambda, d, \ell, \log N) \quad \text{and} \quad |\mathsf{hsk}_i| = \mathrm{poly}(\lambda, d, \ell, \log N) \quad \forall i \in [N].$$

Security. For a security parameter λ and a challenge bit $b \in \{0,1\}$, the security game between an adversary $\mathcal{A}$ and a challenger is defined as follows:

- **Initialization.** The adversary $\mathcal{A}$ sends a slot count 1^N and policy parameters $1^d, 1^\ell$ to the challenger. The challenger samples a common reference string $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^N, 1^d, 1^\ell)$ and sends crs to $\mathcal{A}$. It also initializes a counter $\mathsf{ctr} \leftarrow 0$, a dictionary $\mathcal{D}$, and a set of slot indices $\mathcal{C} \leftarrow \varnothing$.
- **Query Phase.** The adversary $\mathcal{A}$ may make the following types of queries:
  - *Honest Key Query:* The adversary $\mathcal{A}$ specifies a slot index $i \in [N]$ and a policy f. The challenger increments the counter $\mathsf{ctr} \leftarrow \mathsf{ctr} + 1$, samples $(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, i, f)$, and sends $(\mathsf{ctr}, \mathsf{pk}_{\mathsf{ctr}})$ to $\mathcal{A}$. The challenger adds the mapping $\mathsf{ctr} \mapsto (i, f, \mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}})$ to the dictionary $\mathcal{D}$.
  - *Corrupted Key Query:* The adversary $\mathcal{A}$ specifies an index $1 \le c \le \mathsf{ctr}$. The challenger looks up the tuple $(i', f', \mathsf{pk}', \mathsf{sk}') \leftarrow \mathcal{D}[c]$ and replies to $\mathcal{A}$ with sk'. The challenger adds the slot index i' to $\mathcal{C}$.
- **Challenge Phase.** First, $\mathcal{A}$ chooses two messages $\mu_0^*, \mu_1^* \in \mathcal{M}$ and a target attribute vector $\mathbf{w}^* \in \mathcal{X}$. Additionally, for each slot $i \in [N]$, $\mathcal{A}$ provides a triple $(c_i,\ f_i^*,\ \mathsf{pk}_i^*)$, where c_i is either an index $1 \le c_i \le \mathsf{ctr}$ referring to a challenger-generated key or the symbol $\perp$ indicating a fresh key choice. The challenger then, for each $i \in [N]$, carries out:
  - If $1 \le c_i \le \mathsf{ctr}$, retrieve $\mathcal{D}[c_i] = (i',\ f',\ \mathsf{pk}',\ \mathsf{sk}')$. If any of $i \ne i'$, $f_i^* \ne f'$, or $\mathsf{pk}_i^* \ne \mathsf{pk}'$ holds, the game halts with output 0. If that key was previously corrupted, include i in the set $\mathcal{C}$.
  - If $c_i = \perp$, invoke $\mathsf{IsValid}\big(\mathsf{crs},\ i,\ f_i^*,\ \mathsf{pk}_i^*\big)$ and abort with 0 if it returns 0; otherwise add i to $\mathcal{C}$.

  The challenger computes $(\mathsf{mpk}, \mathsf{hsk}_1, \ldots, \mathsf{hsk}_N) \leftarrow \mathsf{Aggregate}(\mathsf{crs}, \mathsf{pk}_1^*, \ldots, \mathsf{pk}_N^*)$, then chooses a bit $b \in \{0,1\}$, and returns $\mathsf{ct}^* = \mathsf{Encrypt}(\mathsf{mpk},\ \mathbf{w}^*,\ \mu_b^*)$ to $\mathcal{A}$.
- **Output Phase.** The adversary $\mathcal{A}$ outputs a guess bit $b' \in \{0,1\}$.

The adversary $\mathcal{A}$ is said to be *admissible* if for all corrupted slot indices $i \in \mathcal{C}$, the challenge attribute set $\mathbf{w}^*$ does not satisfy any policy f_i^*. We say that the slotted registered ABE scheme is secure if for all valid and efficient adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left|\Pr\left[b' = 1 \mid b = 0\right] - \Pr\left[b' = 1 \mid b = 1\right]\right| \le \mathsf{negl}(\lambda).$$

The public keys $\mathsf{pk}_1, \ldots, \mathsf{pk}_N$ in the above security game are classified into three types:

- **Honest User:** Remaining pk_i are honest; the challenger knows sk_i but the adversary does not.
- **Corrupted User:** pk_i with slot index $i \in \mathcal{C}$; both the challenger and the adversary know sk_i.
- **Malicious User:** pk_i without a corresponding sk_i in $\mathcal{D}$; the challenger does not know sk_i.

Remark 2.12 (Differences from [HLWW23]) *In the ciphertext-policy registered ABE proposed by [HLWW23], the key generation algorithm does not take attributes as input. In contrast, in our key-policy registered ABE, the policy is provided during the key generation phase, meaning that the generated public key is associated with the policy. We believe this modification does not affect the overall correctness and security of the scheme, as it still preserves the core requirement of registered ABE, namely that users can independently choose their own keys without relying on any trusted authority.*

Definition 2.13 (Selective Security) *We say that a key-policy slotted registered ABE scheme satisfies selective security if the adversary is required to claim its attribute* $\mathbf{w}$ *at the beginning of the setup phase before seeing the common reference string.*

3 Slotted Key-Policy Registered ABE for Circuits

Overview. In our slotted key-policy registered ABE scheme, we first fix a priori number of slots N. We use $\{ind^{(i)}\}_{i\in[N]}$ to denote the set of all binary strings of length $k = \log N$, where $ind^{(i)} \in \{0,1\}^k$ is the binary representation of $i-1$. The construction of the scheme is parameterized by $n, m, p, q \in \mathbb{N}$ and distributions χ_1, χ_2, γ over $\mathbb{Z}$, where ℓ denotes the length of attribute vectors, $\mathbf{y}_\epsilon$ is the label of the root node ϵ. Each secret and public keys takes the form of $(\mathsf{sk}_i = \mathbf{x}_i = \mathbf{x}'_i + \mathbf{x}''_i, \mathsf{pk}_i = \mathbf{y}_i) \in \mathbb{Z}_p^{2m} \times \mathbb{Z}_q^n$, satisfying $(\mathbf{B} \mid \mathbf{C}_f) \cdot \mathbf{x}_i = \mathbf{y}_i \pmod q$. We define the NP relation $\mathcal{C}_\mathcal{R}$ for the following relation that checks knowledge of a secret key associated with a public key:

- **Statement:** matrices $\mathbf{B}, \mathbf{C} \in \mathbb{Z}_q^{n\times m}$, vectors $\mathbf{x}' \in \mathbb{Z}_q^{2m}, \mathbf{y} \in \mathbb{Z}_q^n$.
- **Witness:** vector $\mathbf{x}'' \in \mathbb{Z}_q^{2m}$.
- Output 1 if $\mathbf{y} = (\mathbf{B} \mid \mathbf{C}) \cdot (\mathbf{x}' + \mathbf{x}'')$. Otherwise, output 0.

We insert the public key pk_i at the corresponding leaf node by $\mathbf{y}_{ind^{(i)}}$ as its label. The Merkle tree aggregation process proceeds as follows. Along the path from the leaf node ind to the root node ϵ, for each node $str \in \{0,1\}^j, j \in [k-1, 0]$, we recursively compute its label:

- Let $\mathbf{y}_{str\|0}$ and $\mathbf{y}_{str\|1}$ denote the labels of the left and right child nodes of str, respectively.
- Compute $\mathbf{u}_{str\|0} \coloneqq -\mathbf{G}^{-1}(\mathbf{y}_{str\|0})$ and $\mathbf{u}_{str\|1} \coloneqq -\mathbf{G}^{-1}(\mathbf{y}_{str\|1})$.
- Compute $\mathbf{y}_{str} \coloneqq \mathbf{A}_0 \cdot \mathbf{u}_{str\|0} + \mathbf{A}_1 \cdot \mathbf{u}_{str\|1}$.
- Assign the value of $\mathbf{y}_{str}$ as the label of node str.

We designate the aggregated label $\mathbf{y}_\epsilon$ of the root node as the master public key. Given an attribute vector $\mathbf{w}$, the master public key mpk, and a message μ, the sender computes the ciphertext as

$$\mathsf{ct} \leftarrow \mathrm{Encrypt}(\mathsf{mpk}, \mathbf{w}, \mu).$$

For correctness, the scheme requires that any registered user whose access policy f satisfies $f(\mathbf{w}) = 0$ can correctly decrypt the ciphertext using their secret key and the helper decryption key:

$$\mu = \mathrm{Decrypt}(\mathsf{sk}_{rcv}, \mathsf{hsk}_{rcv}, \mathsf{ct}),$$

where hsk_{rcv} is the helper decryption key generated by the KC according to the receiver's slot index $rcv \in [N]$. The hsk consists of the Merkle tree opening (i.e., the root-to-leaf path corresponding to the key pk_{rcv}) and some slot-specific components generated in the Setup algorithm.

3.1 Our Construction

We construct a slotted registered key-policy attribute-based encryption scheme $\Pi_{\mathsf{sRABE}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Aggregate}, \mathsf{Encrypt}, \mathsf{Decrypt})$ as follows:

$\mathsf{Setup}(1^\lambda, 1^N, 1^\ell, 1^d)$: On input the security parameter λ, the bound on the number of slots N, and the circuit parameter ℓ, d, the setup algorithm proceeds as follows:

1. Sample random matrices $\mathbf{A}_0 \leftarrow \mathbb{Z}_q^{n\times m}$, $\mathbf{A}_1 \leftarrow \mathbb{Z}_q^{n\times m}$, and $\mathbf{B} \leftarrow \mathbb{Z}_q^{n\times m}$. Use the $\mathsf{TrapGen}$ algorithm to sample $(\mathbf{D}, \mathbf{D}_\tau^{-1}) \leftarrow \mathsf{TrapGen}(1^{(k+1)nm}, q)$.

   Attribute-specific Component. Sample ℓ random matrices $\mathbf{C}_c \leftarrow \mathbb{Z}_q^{n\times m}$ for each $c \in [\ell]$, and set $\mathbf{C} = (\mathbf{C}_1, \ldots, \mathbf{C}_\ell)$.

2. Each registered user $i \in [N]$ corresponds to a leaf node in the tree indexed by $ind^{(i)} \in \{0,1\}^k$. For each index $ind^{(i)}$, compute the corresponding matrix $\mathbf{T}_i \in \mathbb{Z}_q^{(k+1)n \times 2km}$ as follows:

$$\mathbf{T}_i = \begin{pmatrix} \mathbf{A}_0 & \mathbf{A}_1 & & & & & \\ \overline{ind}_1^{(i)} \cdot \mathbf{G} & ind_1^{(i)} \cdot \mathbf{G} & \mathbf{A}_0 & \mathbf{A}_1 & & & \\ & & \overline{ind}_2^{(i)} \cdot \mathbf{G} & ind_2^{(i)} \cdot \mathbf{G} & & & \\ & & & & \ddots & \ddots & \\ & & & & & \mathbf{A}_0 & \mathbf{A}_1 \\ & & & & & \overline{ind}_k^{(i)} \cdot \mathbf{G} & ind_k^{(i)} \cdot \mathbf{G} \end{pmatrix}$$

Slot-specific Component. Sample random vectors $\mathbf{r}_i \leftarrow \gamma^m$ and $\mathbf{x}'_i \leftarrow \{0,1\}^{2m}$ for $i \in [N]$. Using the trapdoor $\mathbf{D}_\tau^{-1}$ and standard deviation $O(\sqrt{(k+1)nm \log q})$, for each $i \in [N]$, sample

$$\mathbf{L}_i \leftarrow \mathbf{D}^{-1}(\mathbf{T}_i \otimes \mathbf{r}_i, \tau)$$

3. Sample $\mathsf{crs}_{\mathsf{NIZK}} \leftarrow \mathsf{NIZK.Setup}(1^\lambda)$.
4. Output $\mathsf{crs} = (\mathsf{crs}_{\mathsf{NIZK}}, \mathbf{A}_0, \mathbf{A}_1, \mathbf{B}, \mathbf{C}, \mathbf{D}, \{\mathbf{L}_i, \mathbf{r}_i, \mathbf{x}'_i\}_{i \in [N]})$.

$\mathsf{KeyGen}(\mathsf{crs}, f, i)$: On input the common reference string crs, a policy f associated with the user, and an index $i \in [N]$. the key-generation algorithm does the following:

1. Sample a random vector $\mathbf{x}''_i \leftarrow \{0,1\}^{2m}$. Compute $\mathbf{x}_i = \mathbf{x}'_i + \mathbf{x}''_i$ and set the secret key $\mathsf{sk}_i := \mathbf{x}_i$.
2. Use the homomorphic evaluation algorithm to compute

$$\mathbf{H}_f \leftarrow \mathsf{EvalF}(\mathbf{C}, f).$$

3. Compute $\mathbf{y}_i \coloneqq (\mathbf{B} \mid \mathbf{C}\mathbf{H}_f) \cdot \mathbf{x}_i$, and construct the NIZK proof

$$\pi_i \leftarrow \mathsf{NIZK.Prove}\left(\mathsf{crs}_{\mathsf{NIZK}}, \mathcal{C}_{\mathcal{R}}, (\mathbf{B}, \mathbf{C}_f, \mathbf{x}'_i, \mathbf{y}_i), \mathbf{x}''_i\right).$$

4. Output $\mathsf{pk}_i := (\mathbf{y}_i, \pi_i)$ and $\mathsf{sk}_i := \mathbf{x}_i$.

$\mathsf{IsValid}(\mathsf{crs}, i, f, \mathsf{pk}_i)$: On input the common reference string crs, an index $i \in [N]$, a policy f, and a public key pk_i, the validity-checking algorithm proceeds as follows:

1. Computes $\mathbf{C}_f = \mathsf{EvalF}(\mathbf{C}, f)$.
2. Output 1 if $\mathsf{NIZK.Verify}(\mathsf{crs}_{\mathsf{NIZK}}, (\mathbf{B}, \mathbf{C}_f, \mathbf{x}'_i, \mathbf{y}_i), \pi_i) = 1$. Otherwise, output 0.

$\mathsf{Aggregate}(\mathsf{crs}, \{\mathsf{pk}_i\}_{i \in [N]})$: On input the common reference string crs and the public keys of N registered users $\{\mathsf{pk}_i\}_{i \in [N]}$, the aggregate algorithm proceeds as follows:

1. For each slot $i \in [N]$, assign a leaf node index $ind^{(i)} \in \{0,1\}^k$ and set $\mathbf{y}_{ind^{(i)}} = \mathsf{pk}_i$.
2. For $j = k-1, \ldots, 0$, recursively compute

$$\mathbf{y}_{str} \coloneqq \mathbf{A}_0 \cdot (-\mathbf{G}^{-1}(\mathbf{y}_{str\|0})) + \mathbf{A}_1 \cdot (-\mathbf{G}^{-1}(\mathbf{y}_{str\|1})), \quad \forall\ str \in \{0,1\}^j$$

3. Output the final root node value $\mathbf{y}_\epsilon$ and set the master public key

$$\mathsf{mpk} \coloneqq (\mathbf{B}, \mathbf{D}, \mathbf{C}, \mathbf{y}_\epsilon).$$

4. We denote the nodes on the path from the leaf node $ind^{(i)}$ to the root as $ind^{(i)}_{1:k}, \ldots, ind^{(i)}_{1:0}$. For each leaf node $ind^{(i)} \in \{0,1\}^k$, output the witness (the Merkle tree opening path from the leaf node to the root):

$$\mathsf{wit}_i \coloneqq \{\mathbf{u}_{ind^{(i)}_{1:j}\|0}, \mathbf{u}_{ind^{(i)}_{1:j}\|1}\}_{j=0}^{k-1}.$$

where

$$\mathbf{u}_{ind^{(i)}_{1:j}\|0} = -\mathbf{G}^{-1}(\mathbf{y}_{ind^{(i)}_{1:j}\|0}), \quad \mathbf{u}_{ind^{(i)}_{1:j}\|1} = -\mathbf{G}^{-1}(\mathbf{y}_{ind^{(i)}_{1:j}\|1})$$

5. For each user $i \in [N]$, output the helper decryption key $\mathsf{hsk}_i = (\mathsf{wit}_i, \mathbf{L}_i, \mathbf{r}_i)$.

$\mathsf{Encrypt}(\mathsf{mpk}, \mathbf{w}, \mu)$: On input the master public key mpk, the sender's attribute set $\mathbf{w} \in \{0,1\}^\ell$, and a message μ, the encrypt algorithm proceeds as follows:

1. For each $j \in [0,k]$, sample a random vector $\mathbf{s}_j \leftarrow \mathbb{Z}_q^{nm}$.
2. Sample $\mathbf{e}_1 \leftarrow \chi_1^{O((k+1)nm\log q)}$, and compute the slot-specific component:
$$\mathbf{c}^\top \coloneqq (\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D} + \mathbf{e}_1^\top$$
3. Sample $\mathbf{e}_2 \leftarrow \chi_2^{m^2}, \mathbf{e}_3 \leftarrow \chi_2^m$, and compute the message-embedding components:
$$\mathbf{c}_k^\top \coloneqq \mathbf{s}_k^\top \cdot (\mathbf{B} \otimes \mathbf{I}_m) + \mathbf{e}_2^\top,$$
$$\mathbf{d}^\top \coloneqq \mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m) + \mathbf{e}_3^\top + \mu \cdot \mathbf{g}$$
4. For attribute $\mathbf{w}$, sample $\mathbf{e}' \leftarrow \chi_2^{m^2\ell}$ and compute the attribute-specific component:
$$\boldsymbol{\psi}^\top \coloneqq \mathbf{s}_k^\top \cdot ((\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \otimes \mathbf{I}_m) + \mathbf{e}'^\top$$
5. Output $\mathsf{ct} \coloneqq (\mathbf{c}, \mathbf{c}_k, \mathbf{d}, \boldsymbol{\psi})$.

$\mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, \mathsf{ct})$: On input the receiver's secret key sk, the helper decryption key hsk, and the ciphertext ct, the decryption algorithm proceeds as follows:

1. Use the homomorphic evaluation algorithm to compute:
$$\mathbf{H}_{\mathbf{C},f,\mathbf{w}} \leftarrow \mathsf{EvalFX}(\mathbf{C}, f, \mathbf{w}).$$
2. Compute:
$$\bar{\mu} = \mathbf{d}^\top \cdot \mathbf{r}_{rcv} - \mathbf{c}^\top \cdot \mathbf{L}_{rcv} \cdot \begin{pmatrix} \mathbf{u}_{ind_{1:0}^{(rcv)}\|0} \\ \mathbf{u}_{ind_{1:0}^{(rcv)}\|1} \\ \vdots \\ \mathbf{u}_{ind_{1:k-1}^{(rcv)}\|0} \\ \mathbf{u}_{ind_{1:k-1}^{(rcv)}\|1} \end{pmatrix} - (\mathbf{c}_k^\top \mid \boldsymbol{\psi}^\top \cdot (\mathbf{H}_{\mathbf{C},f,\mathbf{w}} \otimes \mathbf{I}_m)) \cdot (\mathbf{x}_{ind^{(rcv)}} \otimes \mathbf{r}_{rcv}).$$
3. If $|\bar{\mu}| < \beta_0$, output 0; otherwise, output 1.

Completeness. We give the proof below that the slotted registered ABE construction is complete if the NIZK proof system Π_{NIZK} is complete.

Let $\mathsf{crs} = (\mathsf{crs}_{\mathsf{NIZK}}, \mathbf{A}_0, \mathbf{A}_1, \mathbf{B}, \mathbf{C}, \mathbf{D}, \{\mathbf{L}_i, \mathbf{r}_i, \mathbf{x}'_i\}_{i\in[N]})$ be the public parameters output by $\mathsf{Setup}(1^\lambda, 1^N, 1^\ell, 1^d)$. Fix any user index $i \in [N]$ and any policy f. In the key generation algorithm:

- A random vector $\mathbf{x}''_i \leftarrow \{0,1\}^{2m}$ is sampled, and $\mathbf{x}_i = \mathbf{x}'_i + \mathbf{x}''_i$ is computed.
- The policy matrix is computed as $\mathbf{C}_f = \mathsf{EvalF}(\mathbf{C}, f)$.
- The public key component is computed as $\mathbf{y}_i = (\mathbf{B} \mid \mathbf{C}_f) \cdot \mathbf{x}_i$.
- The NIZK proof is generated as $\pi_i \leftarrow \mathsf{NIZK.Prove}(\mathsf{crs}_{\mathsf{NIZK}}, \mathcal{C}_{\mathcal{R}}, (\mathbf{B}, \mathbf{C}_f, \mathbf{x}'_i, \mathbf{y}_i), \mathbf{x}''_i)$, where the relation $\mathcal{C}_{\mathcal{R}}$ checks whether $\mathbf{y}_i = (\mathbf{B} \mid \mathbf{C}_f) \cdot (\mathbf{x}'_i + \mathbf{x}''_i)$.

Since the witness $\mathbf{x}''_i$ and the instance $(\mathbf{B}, \mathbf{C}_f, \mathbf{x}'_i, \mathbf{y}_i)$ satisfy the relation $\mathcal{C}_{\mathcal{R}}$ by construction, the completeness of Π_{NIZK} ensures that
$$\mathsf{NIZK.Verify}(\mathsf{crs}_{\mathsf{NIZK}}, \mathcal{C}_{\mathcal{R}}, (\mathbf{B}, \mathbf{C}_f, \mathbf{x}'_i, \mathbf{y}_i), \pi_i) = 1.$$
Therefore, $\mathsf{IsValid}(\mathsf{crs}, i, f, \mathsf{pk}_i) = 1$ and the claim holds.

Parameters. Suppose $|\mathbf{H}_{\mathbf{C},f,\mathbf{w}}|$ is bounded by β. We set
$$n = \mathrm{poly}(\lambda, d), \quad m = O(n\log q), \quad \tau = O(\sqrt{(k+1)nm\log q})$$
$$\beta = m^{O(d)}, \quad \gamma, \chi_2 = \lambda^{\omega(1)}, \quad \chi_1 = \chi_2\beta\gamma \cdot \lambda^{\omega(1)}$$
$$\beta_0 = \beta\tau\chi_1\chi_2\gamma \cdot \lambda^{\omega(1)}, \quad q = \beta_0 \cdot \lambda^{\omega(1)}$$

Correctness. We first observe that $\mathbf{T}_{rcv} \cdot \mathsf{wit}_{rcv} = (\mathbf{y}_\epsilon, \mathbf{0}, \ldots, \mathbf{0}, -\mathbf{y}_{ind^{(rcv)}})^\top$, thus we have

$$\begin{aligned}
\mathbf{c}^\top \cdot \mathbf{L}_{rcv} \cdot \mathsf{wit}_{rcv} &= (\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D} \cdot \mathbf{D}^{-1}(\mathbf{T}_{rcv} \otimes \mathbf{r}_{rcv}) \cdot \mathsf{wit}_{rcv} + \overline{e}_1 \\
&= (\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot (\mathbf{T}_{rcv} \cdot \mathsf{wit}_{rcv}) \otimes \mathbf{r}_{rcv} + \overline{e}_1 \\
&= (\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot (\mathbf{T}_{rcv} \cdot \mathsf{wit}_{rcv} \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} + \overline{e}_1 \\
&= \mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} - \mathbf{s}_k^\top \cdot (\mathbf{y}_{ind^{(rcv)}} \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} + \overline{e}_1
\end{aligned}$$

According to the equation $(\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbf{C},f,\mathbf{w}} = \mathbf{C}_f - f(\mathbf{w})\mathbf{G}$, if $f(\mathbf{w}) = 0$, it follows that

$$\begin{aligned}
&(\mathbf{c}_k^\top \mid \boldsymbol{\psi}^\top \cdot (\mathbf{H}_{\mathbf{C},f,\mathbf{w}} \otimes \mathbf{I}_m)) \cdot (\mathbf{x}_{ind^{(rcv)}} \otimes \mathbf{r}_{rcv}) \\
&= \left(\mathbf{s}_k^\top \cdot (\mathbf{B} \otimes \mathbf{I}_m) + \mathbf{e}_2^\top \mid \mathbf{s}_k^\top \cdot ((\mathbf{C}_f - f(\mathbf{w})\mathbf{G}) \otimes \mathbf{I}_m) + \mathbf{e}_f'^\top\right) \cdot (\mathbf{x}_{ind^{(rcv)}} \otimes \mathbf{r}_{rcv}) \\
&= \mathbf{s}_k^\top \cdot ((\mathbf{B} \mid \mathbf{C}_f) \cdot \mathbf{x}_{ind^{(rcv)}} \otimes \mathbf{r}_{rcv}) + (\mathbf{e}_2^\top \mid \mathbf{e}_f'^\top) \cdot (\mathbf{x}_{ind^{(rcv)}} \otimes \mathbf{r}_{rcv}) \\
&= \mathbf{s}_k^\top \cdot (\mathbf{y}_{ind^{(rcv)}} \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} + \overline{e}_2
\end{aligned}$$

In addition, we have $\mathbf{d} \cdot \mathbf{r}_{rcv} = \mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} + \mu \cdot \mathbf{g} \cdot \mathbf{r}_{rcv} + \overline{e}_3$. During decryption, we obtain

$$\begin{aligned}
&\mathbf{d} \cdot \mathbf{r}_{rcv} - \mathbf{c}^\top \cdot \mathbf{L}_{rcv} \cdot \mathsf{wit}_{rcv} - (\mathbf{c}_k^\top \mid \boldsymbol{\psi}^\top \cdot (\mathbf{H}_{\mathbf{C},f,\mathbf{w}} \otimes \mathbf{I}_m)) \cdot (\mathbf{x}_{ind^{(rcv)}} \otimes \mathbf{r}_{rcv}) \\
&= \mathbf{s}_0^\top (\mathbf{y}_\epsilon \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} + \mu \cdot \mathbf{g} \cdot \mathbf{r}_{rcv} - \mathbf{s}_0^\top (\mathbf{y}_\epsilon \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} + \mathbf{s}_k^\top (\mathbf{y}_{ind^{(rcv)}} \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} - \mathbf{s}_k^\top (\mathbf{y}_{ind^{(rcv)}} \otimes \mathbf{I}_m) \cdot \mathbf{r}_{rcv} + \overline{e}_4 \\
&= \mu \cdot \mathbf{g} \cdot \mathbf{r}_{rcv} + \overline{e}_4
\end{aligned}$$

In particular, the error term is bounded by

$$\begin{aligned}
\|\overline{e}_4\|_\infty &= \|\overline{e}_3\|_\infty + \|\overline{e}_1\|_\infty + \|\overline{e}_2\|_\infty \\
&= \left\|\mathbf{e}_3^\top \cdot \mathbf{r}_{ind^{(rcv)}}\right\|_\infty + \left\|\mathbf{e}_1^\top \cdot \mathbf{L}_{rcv} \cdot \mathsf{wit}_{rcv}\right\|_\infty + \left\|(\mathbf{e}_2^\top \mid \mathbf{e}_f'^\top) \cdot (\mathbf{x}_{ind^{(rcv)}} \otimes \mathbf{r}_{rcv})\right\|_\infty \\
&\leq (\chi_2\gamma + \chi_1\tau + 2\chi_2\beta\gamma) \cdot \mathrm{poly}(m) \\
&\leq \beta_0
\end{aligned}$$

Now, $\mathbf{g} \cdot \mathbf{r}_{rcv}$ is statistically close to uniform over $\mathbb{Z}_q$, and correctness follows as long as $q \geq \beta_0 \cdot \lambda^{\omega(1)}$.

3.2 Security

Here, we prove the following theorem, which asserts the security of our scheme.

Theorem 3.1 *Under the evasive LWE assumption, the tensor LWE assumption, and the standard LWE assumption, our slotted registered ABE scheme supporting policy family $\mathcal{P}$ achieves selective security.*

Proof. To prove the security of the scheme, we assume that the adversary $\mathcal{A}$ is equipped with randomness $\mathsf{coins}_{\mathcal{A}}$. Then, we apply the evasive LWE assumption and define the following sampler Samp, which outputs $(\mathbf{P}, \mathsf{aux})$ as follows:

$$\mathsf{aux} = \left(\mathsf{coins}_{\mathcal{A}}, \mathbf{A}_0, \mathbf{A}_1, \mathbf{B}, \mathbf{C}, \mathbf{y}_\epsilon, \left\{\mathbf{r}_i, ind^{(i)}, \{\mathbf{u}_{ind_{1:j}^{(i)}\|0}, \mathbf{u}_{ind_{1:j}^{(i)}\|1}\}_{j=0}^{k-1}\right\}_{i\in[N]}\right)$$

$$\mathbf{P}_1 = \left(\begin{pmatrix} \mathbf{A}_0 & \mathbf{A}_1 \\ \overline{ind}_1^{(1)} \cdot \mathbf{G} & ind_1^{(1)} \cdot \mathbf{G} \end{pmatrix} \otimes \mathbf{r}_1 \mid \ldots \mid \begin{pmatrix} \mathbf{A}_0 & \mathbf{A}_1 \\ \overline{ind}_1^{(N)} \cdot \mathbf{G} & ind_1^{(N)} \cdot \mathbf{G} \end{pmatrix} \otimes \mathbf{r}_N\right)$$

$$\vdots$$

$$\mathbf{P}_k = \left(\begin{pmatrix} \mathbf{A}_0 & \mathbf{A}_1 \\ \overline{ind}_k^{(1)} \cdot \mathbf{G} & ind_k^{(1)} \cdot \mathbf{G} \end{pmatrix} \otimes \mathbf{r}_1 \mid \ldots \mid \begin{pmatrix} \mathbf{A}_0 & \mathbf{A}_1 \\ \overline{ind}_k^{(N)} \cdot \mathbf{G} & ind_k^{(N)} \cdot \mathbf{G} \end{pmatrix} \otimes \mathbf{r}_N\right)$$

where $\mathbf{P}_i = \begin{pmatrix} \mathbf{P}_{i,1} & \mathbf{P}_{i,2} \\ \mathbf{P}_{i,3} & \mathbf{P}_{i,4} \end{pmatrix}$ can be viewed as consisting of four submatrices of size $nm \times mN$. The construction of $\mathbf{P}$ is as follows:

$$\mathbf{P} = \begin{pmatrix} \mathbf{P}_{1,1} & \mathbf{P}_{1,2} & & & & & \\ \mathbf{P}_{1,3} & \mathbf{P}_{1,4} & \mathbf{P}_{2,1} & \mathbf{P}_{2,2} & & & \\ & & \mathbf{P}_{2,3} & \mathbf{P}_{2,4} & & & \\ & & & & \ddots & \ddots & \\ & & & & & \mathbf{P}_{k,1} & \mathbf{P}_{k,2} \\ & & & & & \mathbf{P}_{k,3} & \mathbf{P}_{k,4} \end{pmatrix}$$

By applying the evasive LWE assumption, we reduce the security of our scheme to a form without short Gaussians. To show that the components of the ciphertext are pseudorandomness, it suffices to prove the indistinguishability of the following two distributions:

Distribution $\mathcal{D}_0$:

$$\begin{pmatrix} \overbrace{(\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D} + \mathbf{e}_1^\top}^{\mathbf{c}^\top}, & \overbrace{\mathbf{s}_k^\top \cdot (\mathbf{B} \otimes \mathbf{I}) + \mathbf{e}_2^\top}^{\mathbf{c}_k^\top}, & \overbrace{\mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}) + \mathbf{e}_3^\top + \mu \cdot \mathbf{g}}^{\mathbf{d}^\top}, & \mathbf{D} \\ \overbrace{\mathbf{s}_k^\top \cdot ((\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \otimes \mathbf{I}) + \mathbf{e}'^\top}^{\boldsymbol{\psi}^\top}, & \left\{(\tilde{\mathbf{c}}_0^{(i)\top}, \ldots, \tilde{\mathbf{c}}_{k-1}^{(i)\top})\right\}_{i \in [N]}, & \left\{c'^{(i)}\right\}_{i \in [N]}, & \mathsf{aux} \end{pmatrix}$$

Distribution $\mathcal{D}_1$:

$$\begin{pmatrix} \mathbf{w}_0^\top, & \mathbf{w}_1^\top, & \mathbf{w}_2^\top, & \mathbf{D} \\ \mathbf{w}_3^\top, & \left\{\mathbf{w}_1'^\top, \ldots, \mathbf{w}_N'^\top\right\}, & \{w_1', \ldots, w_N'\}, & \mathsf{aux} \end{pmatrix}$$

In distribution $\mathcal{D}_1$, all vectors have the same dimensions as the corresponding vectors in $\mathcal{D}_0$, and are sampled uniformly at random from their respective domains. In distribution $\mathcal{D}_0$, we define $\tilde{\mathbf{c}}^{(i)\top} = (\tilde{\mathbf{c}}_0^{(i)\top}, \ldots, \tilde{\mathbf{c}}_{k-1}^{(i)\top})$ and $c'^{(i)} = \tilde{\mathbf{c}}^{(i)\top} \cdot \mathsf{wit}_i$, where $\tilde{\mathbf{c}}^{(i)}$ is given as follows:

$$\tilde{\mathbf{c}}^{(i)\top} \approx (\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \left(\begin{pmatrix} \mathbf{A}_0 & \mathbf{A}_1 & & & & & & \\ \overline{ind}_1^{(i)} \cdot \mathbf{G} & ind_1^{(i)} \cdot \mathbf{G} & \mathbf{A}_0 & \mathbf{A}_1 & & & & \\ & & \overline{ind}_2^{(i)} \cdot \mathbf{G} & ind_2^{(i)} \cdot \mathbf{G} & & & & \\ & & & & \ddots & \ddots & & \\ & & & & & & \mathbf{A}_0 & \mathbf{A}_1 \\ & & & & & & \overline{ind}_k^{(i)} \cdot \mathbf{G} & ind_k^{(i)} \cdot \mathbf{G} \end{pmatrix} \otimes \mathbf{r}_i \right)$$

We denote the vectors of size $2mk$ obtained above as k vectors of size $2m$, denoted by $\tilde{\mathbf{c}}_0^{(i)\top}, \ldots, \tilde{\mathbf{c}}_{k-1}^{(i)\top}$, which are defined as follows:

$$\begin{aligned} \tilde{\mathbf{c}}_j^{(i)\top} &= \mathbf{s}_j^\top \cdot ((\mathbf{A}_0 \mid \mathbf{A}_1) \otimes \mathbf{r}_i) + \mathbf{s}_{j+1}^\top \cdot ((\overline{ind}_{j+1}^{(i)} \cdot \mathbf{G} \mid ind_{j+1}^{(i)} \cdot \mathbf{G}) \otimes \mathbf{r}_i) + \mathbf{e}_{1,j}'^{(i)\top} \\ &= \mathbf{s}_j^\top (\mathbf{I}_n \otimes \mathbf{r}_i) \cdot (\mathbf{A}_0 \mid \mathbf{A}_1) + \mathbf{s}_{j+1}^\top (\mathbf{I}_n \otimes \mathbf{r}_i) \cdot (\overline{ind}_{j+1}^{(i)} \cdot \mathbf{G} \mid ind_{j+1}^{(i)} \cdot \mathbf{G}) + \mathbf{e}_{1,j}'^{(i)\top} \end{aligned}$$

where $j \in [0, k-1]$. If and only if the index-related matrix $\mathbf{T}_i$ is equal to the index value in the helper decryption key $\{\mathbf{u}_{ind_{1:j}^{(i')}\|0}, \mathbf{u}_{ind_{1:j}^{(i')}\|1}\}_{j=0}^{k-1}$; that is, when $i = i'$, the following equation holds:

$$\begin{pmatrix} \mathbf{A}_0 & \mathbf{A}_1 \\ \overline{ind}_j^{(i)} \cdot \mathbf{G} & ind_j^{(i)} \cdot \mathbf{G} \end{pmatrix} \cdot \begin{pmatrix} \mathbf{u}_{ind_{1:j}^{(i)}\|0} \\ \mathbf{u}_{ind_{1:j}^{(i)}\|1} \end{pmatrix} \equiv \begin{pmatrix} \mathbf{y}_{ind_{1:j}^{(i)}} \\ -\mathbf{y}_{ind_{1:j+1}^{(i)}} \end{pmatrix}$$

It can be observed that

$$\tilde{\mathbf{c}}_j^{(i)\top} \cdot \begin{pmatrix} \mathbf{u}_{ind_{1:j}^{(i)}\|0} \\ \mathbf{u}_{ind_{1:j}^{(i)}\|1} \end{pmatrix} = (\mathbf{s}_j^\top \mid \mathbf{s}_{j+1}^\top)(\mathbf{I}_{2n} \otimes \mathbf{r}_i) \cdot \begin{pmatrix} \mathbf{y}_{ind_{1:j}^{(i)}} \\ -\mathbf{y}_{ind_{1:j+1}^{(i)}} \end{pmatrix} + \mathbf{e}_{1,j}'^{(i)\top} \cdot \begin{pmatrix} \mathbf{u}_{ind_{1:j}^{(i)}\|0} \\ \mathbf{u}_{ind_{1:j}^{(i)}\|1} \end{pmatrix}$$

By summing over $j \in [0, k-1]$ from the above expression, we obtain

$$\sum_{j=0}^{k-1} \tilde{\mathbf{c}}_j^{(i)\top} \cdot \begin{pmatrix} \mathbf{u}_{ind_{1:j}^{(i)}\|0} \\ \mathbf{u}_{ind_{1:j}^{(i)}\|1} \end{pmatrix} = \sum_{j=0}^{k-1} (\mathbf{s}_j^\top \mid \mathbf{s}_{j+1}^\top)(\mathbf{I}_{2n} \otimes \mathbf{r}_i) \cdot \begin{pmatrix} \mathbf{y}_{ind_{1:j}^{(i)}} \\ -\mathbf{y}_{ind_{1:j+1}^{(i)}} \end{pmatrix} + \sum_{j=0}^{k-1} \mathbf{e}_{1,j}^{\prime(i)\top} \cdot \begin{pmatrix} \mathbf{u}_{ind_{1:j}^{(i)}\|0} \\ \mathbf{u}_{ind_{1:j}^{(i)}\|1} \end{pmatrix}$$
$$= \mathbf{s}_0^\top (\mathbf{I}_n \otimes \mathbf{r}_i) \cdot \mathbf{y}_\epsilon - \mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i) \cdot \mathbf{y}_{ind_{1:k}^{(i)}} + \sum_{j=0}^{k-1} \mathbf{e}_{1,j}^{\prime(i)\top} \cdot \begin{pmatrix} \mathbf{u}_{ind_{1:j}^{(i)}\|0} \\ \mathbf{u}_{ind_{1:j}^{(i)}\|1} \end{pmatrix}$$

When the index values associated with $\mathbf{T}_i$ and $\mathsf{wit}_{i'}$ are not equal, i.e., $i \neq i'$, there exist extra terms in $\tilde{\mathbf{c}}^{(i)} \cdot \mathsf{wit}_{i'}$ of the following form:

$$\sum_{j=1}^{k-1} \mathbf{s}_j^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i) \cdot \left(\mathbf{y}_{ind_{1:j}^{(i)}} - \mathbf{y}_{ind_{1:j-1}^{(i)} \| \overline{ind}_j^{(i)}} \right)$$

According to leftover hash lemma, the vector $\mathbf{y}$ is indistinguishable from uniform. Therefore, in the following security analysis, we only consider the case where $i = i'$, and define $c'^{(i)} \coloneqq \tilde{\mathbf{c}}^{(i)} \cdot \mathsf{wit}_i$. This is because if security holds in the case $i = i'$, it naturally holds in the case $i \neq i'$ as well. We will analyze the security under two types of queries: corrupted key registration query and honest key registration query. Our goal is to prove the indistinguishability of $\mathcal{D}_0$ and $\mathcal{D}_1$ by constructing a sequence of hybrids.

Corrupted key registration query. We first analyze the security of our slotted registered ABE scheme in the presence of corrupted keys. These include keys honestly generated by the challenger and later corrupted, or keys maliciously registered by the adversary. In both cases, the adversary knows the corresponding secret key $\mathsf{sk} \coloneqq \mathbf{x}$ for some public key $\mathsf{pk} \coloneqq \mathbf{y}$ and policy f^*. According to the admissibility constraint of the slotted registered ABE adversary, it is required that $f^*(\mathbf{w}) = 1$.

- $\mathcal{H}_0$: This is identical to $\mathcal{D}_0$. We have $\mathsf{aux} = (\mathsf{coins}_{\mathcal{A}}, \mathbf{A}_0, \mathbf{A}_1, \mathbf{B}, \mathbf{C}, \mathbf{y}_\epsilon, \{\mathbf{r}_i, ind^{(i)}, \mathsf{wit}_i\}_{i \in [N]})$ and $\mathbf{D}$. The remaining part is structured as follows:

$$\begin{aligned}
\mathbf{c}^\top &\coloneqq (\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot \mathbf{D} + \mathbf{e}_1^\top, \\
\mathbf{c}_k^\top &\coloneqq \mathbf{s}_k^\top \cdot (\mathbf{B} \otimes \mathbf{I}_m) + \mathbf{e}_2^\top, \\
\mathbf{d}^\top &\coloneqq \mathbf{s}_0^\top \cdot (\mathbf{y}_\epsilon \otimes \mathbf{I}_m) + \mathbf{e}_3^\top, \\
\boldsymbol{\psi}^\top &\coloneqq \mathbf{s}_k^\top \cdot ((\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \otimes \mathbf{I}_m) + \mathbf{e}'^\top, \\
\{\tilde{\mathbf{c}}^{(i)\top} &\coloneqq (\mathbf{s}_0^\top, \ldots, \mathbf{s}_k^\top) \cdot (\mathbf{T}_i \otimes \mathbf{r}_i) + \mathbf{e}_1'^\top,\ \mathbf{r}_i^\top\}_{i \in [N]}, \\
\{c'^{(i)} &\coloneqq \mathbf{s}_0^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_\epsilon - \mathbf{s}_k^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_i + \overline{e}_1,\ \mathbf{r}_i^\top\}_{i \in [N]}
\end{aligned}$$

  Here, $\mathbf{c}$, $\mathbf{c}_k$, $\mathbf{d}$, and $\boldsymbol{\psi}$ correspond to the challenge ciphertext components. For simplicity, we omit the term $\mu \cdot \mathbf{g}$ in $\mathbf{d}$. In addition, $\tilde{\mathbf{c}}^{(i)}$ is obtained by removing Gaussian preimage sampling with respect to $\mathbf{D}$. We further compute $c'^{(i)} = \tilde{\mathbf{c}}^{(i)\top} \cdot \mathsf{wit}_i$, and decompose $\tilde{\mathbf{c}}^{(i)\top} = (\tilde{\mathbf{c}}_0^{(i)\top}, \ldots, \tilde{\mathbf{c}}_{k-1}^{(i)\top})$ as a concatenation of k vectors of dimension $2m$, where

$$\tilde{\mathbf{c}}_j^{(i)\top} = \underbrace{\left(\mathbf{s}_j^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{A}_0 \mid \mathbf{A}_1) + \mathbf{e}_{1,j}^{\prime(i)\top} \right)}_{\coloneqq \tilde{\mathbf{c}}_j^{\prime(i)\top}} + \mathbf{s}_{j+1}^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\overline{ind}_{j+1}^{(i)} \cdot \mathbf{G} \mid ind_{j+1}^{(i)} \cdot \mathbf{G})$$

- $\mathcal{H}_1$: same as $\mathcal{H}_0$, except in the key-generation phase, after generating the public key pk_i with policy f_i, the challenger halts with output 0 if $\mathsf{IsValid}(\mathsf{crs}, i, f_i, \mathsf{pk}_i) = 0$.
- $\mathcal{H}_2$: same as $\mathcal{H}_1$, except the simulator $\mathsf{Z} = (\mathsf{Z}_0, \mathsf{Z}_1)$ is employed to compute ζ and π. Specifically, during the setup phase, the simulation algorithm $\mathsf{Z}_0(1^\lambda)$ is employed to generate $(\mathsf{crs}_{\mathsf{NIZK}}, \zeta) \leftarrow \mathsf{Z}_0(1^\lambda)$, and during the key query, the secret key and public key as the rule in $\mathcal{H}_1$, and then the proof is computed as $\pi_i^* = \mathsf{Z}_1(\zeta, \mathcal{C}_{\mathcal{R}}, (\mathbf{B}, \mathbf{C}_f, \mathbf{x}_i', \mathbf{y}_i))$.
- $\mathcal{H}_3$: same as $\mathcal{H}_2$, except we compute

$$c'^{(i)} = \mathbf{d}^\top \cdot \mathbf{r}_i - (\mathbf{c}_k^\top \mid \boldsymbol{\psi}^\top \cdot \mathbf{H}_{\mathbf{C}, f_i, \mathbf{w}}) \cdot (\mathbf{x}_i \otimes \mathbf{r}_i^\top) - \underbrace{\left(\mathbf{s}_k^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{0} \mid \mathbf{G}) \mathbf{x}_i + \overline{e}_1 \right)}_{\coloneqq c''^{(i)\top}}$$

– $\mathcal{H}_4$: same as $\mathcal{H}_3$, except we set

$$\mathbf{C} = \mathbf{B} \cdot \mathbf{R} + \mathbf{w} \otimes \mathbf{G} \in \mathbb{Z}_q^{n \times m\ell},$$

where $\mathbf{R} \leftarrow \{-1, 1\}^{m \times m\ell}$ is sampled uniformly at random. The value of $\boldsymbol{\psi}$ is computed as

$$\boldsymbol{\psi}^\top = \mathbf{c}_k^\top \cdot (\mathbf{R} \otimes \mathbf{I}_m)$$

– $\mathcal{H}_5$: same as $\mathcal{H}_4$, except we sample $\mathbf{c} \leftarrow \mathbb{Z}_q^{O((k+1)nm \log q)}$, $\mathbf{d} \leftarrow \mathbb{Z}_q^m$, $\mathbf{c}_k \leftarrow \mathbb{Z}_q^{m^2}$.

– $\mathcal{H}_6$: same as $\mathcal{H}_5$, except we sample $\boldsymbol{\psi} \leftarrow \mathbb{Z}_q^{m^2\ell}$.

– $\mathcal{H}_7$: same as $\mathcal{H}_6$, except we sample $c'^{(i)} \leftarrow \mathbb{Z}_q$ and $\tilde{\mathbf{c}}_j'^{(i)} \leftarrow \mathbb{Z}_q^{2m}$ for $j \in [0, k-1]$.

– $\mathcal{H}_8$: same as $\mathcal{H}_7$, except we sample $c'^{(i)} \leftarrow \mathbb{Z}_q$, $\tilde{\mathbf{c}}_j^{(i)} \leftarrow \mathbb{Z}_q^{2km}$.

It is easy to see that the distribution in $\mathcal{H}_8$ is the same as that of $\mathcal{D}_1$.

Indistinguishability of hybrids: We prove the indistinguishability between the hybrid distributions via the following claims.

Claim 3.2 $\mathcal{H}_0 \equiv \mathcal{H}_1$

Proof. Immediate since the construction satisfies (perfect) completeness.

Claim 3.3 $\mathcal{H}_1 \approx_c \mathcal{H}_2$

Proof. Review that in $\mathcal{H}_1$, the proof $\pi_i \leftarrow \mathsf{NIZK.Prove}(\mathsf{crs}_{\mathsf{NIZK}}, \mathcal{C}_{\mathcal{R}}, (\mathbf{B}, \mathbf{C}_f, \mathbf{x}_i', \mathbf{y}_i), \mathbf{x}_i'')$ is created via the real algorithms, and in $\mathcal{H}_2$, it is computed via the simulation algorithms $(\mathsf{crs}_{\mathsf{NIZK}}, \zeta) \leftarrow \mathsf{Z}_0(1^\lambda)$ and $\pi_i^* = \mathsf{Z}_1(\zeta, \mathcal{C}_{\mathcal{R}}, (\mathbf{B}, \mathbf{C}_f, \mathbf{x}_i', \mathbf{y}_i))$. Suppose that we have a PPT adversary $\mathcal{A}$ that is capable of distinguishing $\mathcal{H}_1$ and $\mathcal{H}_2$. If so, we are able to construct an efficient algorithm $\mathcal{B}$ to violate the zero-knowledge property of the NIZK. We will now explain the reduction process in greater detail:

– $\mathcal{B}$ constructs crs using same procedure as described in $\mathcal{H}_1$ and $\mathcal{H}_2$, except it uses $\mathsf{crs}_{\mathsf{NIZK}}$ from the challenger. It gives crs to $\mathcal{A}$.
– Whenever $\mathcal{B}$ makes a key-generation query (i, f_i), $\mathcal{B}$ samples $\mathbf{y}_i$ as in $\mathcal{H}_1$ and $\mathcal{H}_2$. To generate the NIZK proof, $\mathcal{B}$ forwards the circuit $\mathcal{C}_{\mathcal{R}}$, the statement $(\mathbf{B}, \mathbf{C}_f, \mathbf{x}_i', \mathbf{y}_i)$, and the witness $\mathbf{x}_i''$ to the zero-knowledge challenger. The challenger replies with a proof π_i^*. Finally, $\mathcal{B}$ assigns the public key to be $\mathsf{pk}_i = (\mathbf{y}_i, \pi_i^*)$ and then transmits it to $\mathcal{A}$.
– $\mathcal{B}$ executes the challenge phase and the output phase exactly as in $\mathcal{H}_1$ and $\mathcal{H}_2$.

We argue that when the proof π_i^* is generated by running the real algorithms, then the distributions of crs and pk_i are as in $\mathcal{H}_1$, otherwise, they are as in $\mathcal{H}_2$. As a result, if $\mathcal{A}$ can differentiate between $\mathcal{H}_1$ and $\mathcal{H}_2$, then $\mathcal{B}$ disobeys the zero-knowledge property of the NIZK proof system. Consequently, $\mathcal{B}$'s success probability is almost negligible compared to that of $\mathcal{A}$'s throughout the entire simulation experiment.

Claim 3.4 $\mathcal{H}_2 \approx_s \mathcal{H}_3$

Proof. The two hybrids differ only in the error term in $c'^{(i)}$ and are indistinguishable due to the smudging lemma.

In $\mathcal{H}_2$:

$$c'^{(i)} = \mathbf{s}_0^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_\epsilon - \mathbf{s}_k^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_i + \overline{e}_1$$

In $\mathcal{H}_3$:

$$\begin{aligned}
c'^{(i)} &= \mathbf{d}^\top \cdot \mathbf{r}_i - (\mathbf{c}_k^\top \mid \boldsymbol{\psi}^\top \cdot \mathbf{H}_{\mathbf{C},f_i,\mathbf{w}}) \cdot (\mathbf{x}_i \otimes \mathbf{r}_i^\top) - \left(\mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{0} \mid \mathbf{G})\mathbf{x}_i + \overline{e}_1\right) \\
&= \mathbf{d}^\top \cdot \mathbf{r}_i - \left(\mathbf{c}_k^\top (\mathbf{I}_m \otimes \mathbf{r}_i^\top) \mid \boldsymbol{\psi}^\top (\mathbf{I}_m \otimes \mathbf{r}_i^\top) \cdot \mathbf{H}_{\mathbf{C},f_i,\mathbf{w}}\right) \cdot \mathbf{x}_i - \left(\mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (0 \mid \mathbf{G})\mathbf{x}_i + \overline{e}_1\right) \\
&= \mathbf{s}_0^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_\epsilon + \mathbf{e}_3^\top \cdot \mathbf{r}_i - \mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{B} \mid (\mathbf{C} - \mathbf{w} \otimes \mathbf{G})\mathbf{H}_{\mathbf{C},f_i,\mathbf{w}} + \mathbf{G}) \cdot \mathbf{x}_i \\
&\quad - (\mathbf{e}_2^\top \mid \mathbf{e}'^\top \cdot \mathbf{H}_{\mathbf{C},f_i,\mathbf{w}}) \cdot \mathbf{r}_i - \overline{e}_1 \\
&= \mathbf{s}_0^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_\epsilon - \mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{B} \mid \mathbf{C}_{f_i}) \cdot \mathbf{x}_i + \mathbf{e}_3^\top \cdot \mathbf{r}_i - (\mathbf{e}_2^\top \mid \mathbf{e}'^\top \mathbf{H}_{\mathbf{C},f_i,\mathbf{w}}) \cdot \mathbf{r}_i - \overline{e}_1 \\
&= \mathbf{s}_0^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_\epsilon - \mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_i + \mathbf{e}_3^\top \cdot \mathbf{r}_i - (\mathbf{e}_2^\top \mid \mathbf{e}'^\top \mathbf{H}_{\mathbf{C},f_i,\mathbf{w}}) \cdot \mathbf{r}_i - \overline{e}_1
\end{aligned}$$

The equality holds due to the condition $(\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \cdot \mathbf{H}_{\mathbf{C},f_i,\mathbf{w}} = \mathbf{C}_{f_i} - f_i(\mathbf{w})\mathbf{G}$ and $f_i(\mathbf{w}) = 1$. It can be observed that the difference between these two hybrids lies only in the noise terms. Therefore, the indistinguishability follows from the following fact:

$$\overline{e}_1 \approx_{\mathrm{s}} -\overline{e}_1 + \mathbf{e}_3^\top \cdot \mathbf{r}_i - (\mathbf{e}_2^\top \mid \mathbf{e}'^\top \cdot \mathbf{H}_{\mathbf{A},f_i,\mathbf{w}}) \cdot \mathbf{r}_i$$

which is true since the distribution of $-\overline{e}_1$ is the same as that of $\overline{e}_1$ by the symmetry of the discrete Gaussian distribution. Moreover, the following condition is satisfied:

$$\chi_1 \geq \chi_2 \beta\gamma \cdot \lambda^{\omega(1)}.$$

Claim 3.5 $\mathcal{H}_3 \approx_s \mathcal{H}_4$

Proof. We first recall the difference between the two hybrids:

- In $\mathcal{H}_3$, the matrices $\mathbf{B} \in \mathbb{Z}_q^{n\times m}$ and $\mathbf{C} \in \mathbb{Z}_q^{n\times m\ell}$ are both sampled uniformly at random. However, in $\mathcal{H}_4$, the matrix $\mathbf{C}$ is defined as
$$\mathbf{C} = \mathbf{B} \cdot \mathbf{R} + \mathbf{w} \otimes \mathbf{G},$$
where $\mathbf{R} \leftarrow \{-1, 1\}^{m\times m\ell}$ is sampled uniformly at random.
- In $\mathcal{H}_3$, the ciphertext component $\boldsymbol{\psi}$ is computed as $\boldsymbol{\psi}^\top = \mathbf{s}_k^\top \cdot ((\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \otimes \mathbf{I}) + \mathbf{e}'^\top$. In $\mathcal{H}_4$, the ciphertext computation is as follows:

$$\begin{aligned}\boldsymbol{\psi}^\top &= \mathbf{c}_k^\top \cdot (\mathbf{R} \otimes \mathbf{I}_m)\\ &= \mathbf{s}_k^\top \cdot ((\mathbf{B} \cdot \mathbf{R}) \otimes \mathbf{I}_m) + \mathbf{e}_2^\top \cdot (\mathbf{R} \otimes \mathbf{I}_m)\\ &= \mathbf{s}_k^\top \cdot ((\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \otimes \mathbf{I}_m) + \mathbf{e}_2^\top \cdot (\mathbf{R} \otimes \mathbf{I}_m)\end{aligned}$$

We claim that the joint distribution of the public parameters and ciphertext components, namely $\{\mathbf{B}, \mathbf{C}, \boldsymbol{\psi}\}$, in the two hybrids is statistically indistinguishable. By leftover hash lemma, letting $\mathbf{R} \leftarrow \{-1, 1\}^{m\times m\ell}$ be sampled uniformly at random, we have

$$\left\{\mathbf{B},\ \mathbf{B} \cdot \mathbf{R} + \mathbf{w} \otimes \mathbf{G},\ \mathbf{e}_2^\top \cdot (\mathbf{R} \otimes \mathbf{I}_m)\right\} \approx_{\mathrm{s}} \left\{\mathbf{B},\ \mathbf{C},\ \mathbf{e}'^\top\right\}.$$

Here, the noise vectors $\mathbf{e}'$ and $\mathbf{e}_2$ are sampled independently from the same error distribution. The ciphertext component $\boldsymbol{\psi}$ is computed by adding $\mathbf{s}_k^\top \cdot ((\mathbf{C} - \mathbf{w} \otimes \mathbf{G}) \otimes \mathbf{I}_m)$ and $\mathbf{e}_2^\top \cdot (\mathbf{R} \otimes \mathbf{I}_m)$. Therefore, the ciphertext component $\boldsymbol{\psi}$ is also statistically indistinguishable between the two hybrids. In conclusion, the joint distribution of the public parameters, noise terms, and ciphertext components is statistically indistinguishable in the two hybrids.

Claim 3.6 $\mathcal{H}_4 \approx_c \mathcal{H}_5$

Proof. Since $\mathbf{y}_\epsilon = \mathbf{A}_0(-\mathbf{G}^{-1}(\mathbf{y}_0)) + \mathbf{A}_1(-\mathbf{G}^{-1}(\mathbf{y}_1))$, we set

$$\mathbf{d}^\top = \mathbf{s}_0^\top \cdot ((\mathbf{A}_0 \cdot \mathbf{r}_0 + \mathbf{A}_1 \cdot \mathbf{r}_1) \otimes \mathbf{I}_m) + \mathbf{e}_3^\top = \mathbf{s}_0^\top (\mathbf{A}_0 \otimes \mathbf{I}_m)(\mathbf{r}_0 \otimes \mathbf{I}_m) + \mathbf{s}_0^\top (\mathbf{A}_1 \otimes \mathbf{I}_m)(\mathbf{r}_1 \otimes \mathbf{I}_m) + \mathbf{e}_3^\top,$$

where $\mathbf{r}_0, \mathbf{r}_1$ are binary vectors. Let us write $\mathbf{D}$ as $(\mathbf{D}_0^\top, \ldots, \mathbf{D}_k^\top)^\top$. Then we can see that the indistinguishability follows from LWE by applying Lemma 2.5, which implies

$$(\mathbf{D}_k, \mathbf{B}, \mathbf{s}_k^\top \mathbf{D}_k + \tilde{\mathbf{e}}_1^\top, \mathbf{s}_k^\top (\mathbf{B} \otimes \mathbf{I}_m) + \mathbf{e}_2^\top) \approx_{\mathrm{c}} (\mathbf{D}_k, \mathbf{B}, \mathbf{w}_1', \mathbf{w}_2')$$

$$(\mathbf{D}_0, \mathbf{A}_0, \mathbf{A}_1, \mathbf{s}_0^\top \mathbf{D}_0 + \overline{\mathbf{e}}_1^\top, \mathbf{s}_0^\top ((\mathbf{A}_0 \cdot \mathbf{r}_0 + \mathbf{A}_1 \cdot \mathbf{r}_1) \otimes \mathbf{I}_m) + \mathbf{e}_3^\top) \approx_{\mathrm{c}} (\mathbf{D}_0, \mathbf{A}_0, \mathbf{A}_1, \mathbf{w}_3', \mathbf{w}_4')$$

where $\mathbf{w}_1', \mathbf{w}_3' \leftarrow \mathbb{Z}_q^{O((k+1)nm\log q)}, \mathbf{w}_2' \leftarrow \mathbb{Z}_q^{m^2}, \mathbf{w}_4' \leftarrow \mathbb{Z}_q^m$, $\overline{\mathbf{e}}_1, \tilde{\mathbf{e}}_1 \leftarrow \chi_1^{O((k+1)nm\log q)}$. Then in $\mathcal{H}_4$,

$$\begin{aligned}(\mathbf{c}^\top, \mathbf{c}_k^\top, \mathbf{d}^\top) &= \left(s_0^\top \mathbf{D}_0 + \ldots + s_k^\top \mathbf{D}_k + \mathbf{e}_1^\top, \mathbf{s}_k^\top (\mathbf{B} \otimes \mathbf{I}_m) + \mathbf{e}_2^\top, \mathbf{s}_0^\top ((\mathbf{A}_0 \cdot \mathbf{r}_0 + \mathbf{A}_1 \cdot \mathbf{r}_1) \otimes \mathbf{I}_m) + \mathbf{e}_3^\top\right)\\ &\approx_{\mathrm{c}} \left(\mathbf{w}_1' + s_1^\top \mathbf{D}_1 + \ldots + s_{k-1}^\top \mathbf{D}_{k-1} + \mathbf{w}_3', \mathbf{w}_2', \mathbf{w}_4'\right) \qquad \text{(from LWE)}\\ &\approx_{\mathrm{s}} (\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3) \qquad \text{where } \mathbf{w}_1 \leftarrow \mathbb{Z}_q^{O((k+1)nm\log q)}, \mathbf{w}_2 \leftarrow \mathbb{Z}_q^{m^2}, \mathbf{w}_3 \leftarrow \mathbb{Z}_q^m\end{aligned}$$

If the adversary can distinguish between $\mathcal{H}_4$ and $\mathcal{H}_5$, then it would break the security of the standard LWE assumption.

Claim 3.7 $\mathcal{H}_5 \approx_s \mathcal{H}_6$

Proof. In $\mathcal{H}_5$, the ciphertext component $\boldsymbol{\psi}$ is computed as $\boldsymbol{\psi}^\top = \mathbf{c}_k^\top \cdot (\mathbf{R} \otimes \mathbf{I}_m)$. However, in $\mathcal{H}_6$, $\boldsymbol{\psi}$ is sampled uniformly at random. The indistinguishability between the two hybrids follows from the leftover hash lemma, because:

- The secret key $\mathbf{x}_i$ does not depend on any information about $\mathbf{R}$.
- The only leakage about $\mathbf{R}$ comes from the master public key component $\mathbf{B} \cdot \mathbf{R}$.

Therefore, for all $\mathbf{B}$ and $\mathbf{c}_k^\top$, we have:

$$\left(\mathbf{B},\ \mathbf{c}_k^\top,\ \mathbf{B} \cdot \mathbf{R},\ \mathbf{c}_k^\top \cdot (\mathbf{R} \otimes \mathbf{I}_m)\right) \approx_{\mathrm{s}} \left(\mathbf{B},\ \mathbf{c}_k^\top,\ \mathbf{B} \cdot \mathbf{R},\ \boldsymbol{\psi}^\top\right),$$

where $\mathbf{R}$ is sampled uniformly at random and the parameter m is sufficiently large, specifically $m = O(n \log q)$.

Claim 3.8 $\mathcal{H}_6 \approx_c \mathcal{H}_7$

Proof. We consider the pseudorandomness of $\tilde{\mathbf{c}}_j^{\prime(i)} \leftarrow \mathbb{Z}_q^{2m}$ for $j \in [0, k-1]$ and $c^{\prime\prime(i)} \leftarrow \mathbb{Z}_q$ individually under the tensor LWE assumption.

First, let $i \in [N]$ and $j \in [0, k-1]$, we can see that the indistinguishability follows from LWE by applying Lemma 2.8, which implies:

$$\left(\mathbf{A}_0, \mathbf{A}_1, \{\mathbf{r}_i^\top\}_i, \left\{\mathbf{s}_j^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{A}_0 \mid \mathbf{A}_1) + \mathbf{e}_{1,j}^{\prime\top}\right\}_{i,j}\right) \approx_{\mathrm{c}} \left(\mathbf{A}_0, \mathbf{A}_1, \{\mathbf{r}_i^\top\}_i,\ \{\mathrm{random}\}_{i,j}\right)$$

For $c^{\prime\prime(i)}$, we note that the secret key is of the form $\mathbf{x}_i = \mathbf{x}_i' + \mathbf{x}_i''$, where $\mathbf{x}_i'$ is a uniformly random bit string sampled during system setup, and $\mathbf{x}_i''$ is chosen during key generation. In the malicious setting, although $\mathbf{x}_i''$ is adversarially chosen, the NIZK soundness ensures that $\mathbf{x}_i'$ remains a fixed random string from setup. Therefore, when analyzing the pseudorandomness of the term $c^{\prime\prime(i)}$, we split it into two components:

$$\mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top)(\mathbf{0} \mid \mathbf{G}) \cdot \mathbf{x}_i' + \overline{e}_1 + \mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top)(\mathbf{0} \mid \mathbf{G}) \cdot \mathbf{x}_i''$$

By applying Lemma 2.9, the indistinguishability follows from the tensor LWE assumption, which implies,

$$\left(\{\mathbf{x}_i', \mathbf{r}_i^\top\}_i, \left\{\mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top)(\mathbf{0} \mid \mathbf{G}) \cdot \mathbf{x}_i' + \overline{e}_1\right\}_i\right) \approx_{\mathrm{c}} \left(\{\mathbf{x}_i', \mathbf{r}_i^\top\}_i,\ \{\mathrm{random}\}_i\right).$$

Since the remaining term involving $\mathbf{x}_i''$ is fixed once the key is registered and independent of the challenge bit, the sum remains indistinguishable from uniform.

If an adversary can distinguish between $\mathcal{H}_6$ and $\mathcal{H}_7$, then it would break the security of the tensor LWE assumption.

Claim 3.9 $\mathcal{H}_7 \equiv \mathcal{H}_8$

Proof. In $\mathcal{H}_7$, the terms $c^{\prime(i)}$ and $\tilde{\mathbf{c}}^{(i)\top}$ are masked by the random terms $c^{\prime\prime(i)}$ and $\{\tilde{\mathbf{c}}_j^{\prime(i)\top}\}_{j\in[0,k-1]}$, respectively. Therefore, the equality holds.

Honest key registration query. We now discuss the security of the RABE scheme under honest key registration queries. In this case, the adversary specifies a policy f, and the challenger samples $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, f)$. In this setting, the challenger only sends pk to the adversary, while the adversary has no knowledge of the corresponding secret key sk.
The security proof requires the following modifications:

- $\mathcal{H}_0'$ is same as $\mathcal{H}_0$. The proofs of indistinguishability between $\mathcal{H}_0', \mathcal{H}_1', \mathcal{H}_2'$ follow exactly as in the corrupted key registration query.
- $\mathcal{H}_3'$: same as $\mathcal{H}_2'$, except that

$$c^{\prime(i)} = \mathbf{d}^\top \cdot \mathbf{r}_i - \underbrace{\mathbf{s}_k^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_i + \overline{e}_1}_{:= c^{\prime\prime\prime(i)\top}}$$

The proof of $\mathcal{H}'_2 \approx_s \mathcal{H}'_3$ is similar to the case of corrupted key registration query. The difference between the two hybrids lies only in the error terms of $c'^{(i)}$. In $\mathcal{H}'_3$, we have:

$$\begin{aligned} c'^{(i)} &= \mathbf{d}^\top \cdot \mathbf{r}_i - \left(\mathbf{s}_k^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_i + \overline{e}_1\right) \\ &= \mathbf{s}_0^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_\epsilon + \mathbf{e}_3^\top \cdot \mathbf{r}_i - \mathbf{s}_k^\top \cdot (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_i - \overline{e}_1 \end{aligned}$$

This is ensured by the symmetry of the discrete Gaussian distribution and $\chi_1 \geq \chi_2 \beta\gamma \cdot \lambda^{\omega(1)}$.

- The proofs of indistinguishability between $\mathcal{H}'_4, \mathcal{H}'_5, \mathcal{H}'_6$ follow exactly the same steps as in the case of corrupted key registration query.
- $\mathcal{H}'_7$: same as $\mathcal{H}'_6$, except that samples

$$c'''^{(i)} \leftarrow \mathbb{Z}_q \quad \text{and} \quad \tilde{\mathbf{c}}_j'^{(i)\top} \leftarrow \mathbb{Z}_q^{2m}.$$

Since $\mathbf{C}_f$ is obtained by taking "small-dimension linear combinations" of ℓ matrices (each sampled uniformly at random from $\mathbb{Z}_q^{n\times m}$) via a P/poly function f, its distribution is statistically indistinguishable from that of a uniformly sampled matrix in $\mathbb{Z}_q^{n\times m}$. By applying Lemma 2.2, we conclude that the public key $\mathbf{y}_i = (\mathbf{B} \mid \mathbf{C}_f) \cdot \mathbf{x}_i$ generated by KeyGen is also indistinguishable from uniform. Under the tensor LWE assumption, we have that for all $i \in [N]$ and $j \in [0, k-1]$,

$$\begin{aligned} &\left(\mathbf{A}_0, \mathbf{A}_1, \left\{\mathbf{y}_i, \mathbf{r}_i^\top\right\}_i,\ \left\{\mathbf{s}_j^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot (\mathbf{A}_0 \mid \mathbf{A}_1) + \mathbf{e}'^\top_{1,j},\ \mathbf{s}_k^\top (\mathbf{I}_n \otimes \mathbf{r}_i^\top) \cdot \mathbf{y}_i + \overline{e}_1\right\}_{i,j}\right) \\ &\approx_{\mathrm{c}} \left(\mathbf{A}_0, \mathbf{A}_1, \left\{\mathbf{y}_i, \mathbf{r}_i^\top\right\}_i,\ \{\text{random},\ \text{random}\}_{i,j}\right) \end{aligned}$$

- $\mathcal{H}'_8$: same as $\mathcal{H}'_7$, except that samples $c'^{(i)} \leftarrow \mathbb{Z}_q$ and $\tilde{\mathbf{c}}^{(i)\top} \leftarrow \mathbb{Z}_q^{2km}$.

This completes the proof.

4 From Slotted Registered ABE to Registered ABE

In this section, we show how to generically transform a slotted registered ABE scheme to a standard registered ABE scheme. Registered ABE imposes an additional efficiency requirement: The number of times any user requests an update to their helper decryption key must be at most polynomial in the logarithm of the total number of users. Specifically, in a slotted registered ABE scheme, each user is allowed to register their public key only once and cannot replace or delete it afterwards. Therefore, if there are N users, there will be exactly N key registration updates. A key efficiency feature of the registered ABE scheme is that during these N registrations, the helper decryption key hsk of each user will be updated at most $\mathrm{poly}(\lambda, \log N)$ times.

Construction. Let $\Pi_{\mathsf{sRABE}} = (\mathsf{sRABE.Setup}, \mathsf{sRABE.KeyGen}, \mathsf{sRABE.IsValid}, \mathsf{sRABE.Aggregate}, \mathsf{sRABE.Encrypt}, \mathsf{sRABE.Decrypt})$ be a slotted registered ABE scheme, with attribute space $\mathcal{X} = \{\mathcal{X}_\lambda\}_{\lambda\in\mathbb{N}}$, policy space $\mathcal{P} = \{\mathcal{P}_\lambda\}_{\lambda\in\mathbb{N}}$, and message space $\mathcal{M} = \{\mathcal{M}_\lambda\}_{\lambda\in\mathbb{N}}$. We present a generic construction that transforms any slotted registered ABE scheme into a registered ABE scheme, over the same attribute space $\mathcal{X}$, policy space $\mathcal{P}$, and message space $\mathcal{M}$. The construction is specified as follows. In the following description, we adopt the following conventions:

- We assume that the bound on the number of users $N = 2^k$ is a power of two.
- The registered ABE scheme incorporates $k+1$ distinct slotted ABE schemes. For each index p in the range $[0, k]$, the p-th scheme is configured with 2^p slots.
- In the following construction we assume that the auxiliary information aux consists of the following components: (i) A dictionary $\mathcal{PK}$ that maps each scheme index $p \in [0, k]$ and each slot index $i \in [2^p]$ to the public key pk assigned to slot i in scheme p. (ii) A dictionary $\mathcal{HSK}$ that maps a scheme index $p \in [0, k]$ and a user index $i \in [N]$ to the helper decryption key associated with scheme p and user i. (iii) The number of the users currently registered, denoted ctr. Throughout the lifetime of the registered ABE system, the value of aux is continuously updated as new users are registered.

We construct our registered ABE scheme as follows:

$\mathsf{Setup}(1^\lambda, 1^N, 1^\ell, 1^d)$: On input the security parameter λ, the bound on the number of slots $N = 2^k$, and the circuit parameter ℓ, d, the setup algorithm runs the setup procedure for $k+1$ copies of the slotted RABE scheme. Specifically, for each $p \in [0, k]$, it samples $\mathsf{crs}_p \leftarrow \mathsf{sRABE.Setup}(1^\lambda, 1^{2^p}, 1^\ell, 1^d)$, and outputs the common reference string $\mathsf{crs} = (\mathsf{crs}_0, \ldots, \mathsf{crs}_k)$.

$\mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, f)$: On input the common reference string $\mathsf{crs} = (\mathsf{crs}_0, \ldots, \mathsf{crs}_k)$ and the auxiliary data $\mathsf{aux} = (\mathcal{I}, \mathcal{PK}, \mathsf{ctr}, \mathsf{mpk})$, the key generation algorithm generates a public/secret key pair for each of the $k+1$ underlying slotted RABE schemes. Specifically, for each $p \in [0, k]$ and an policy set f_p, let $i_p = (\mathsf{ctr} \bmod 2^p) + 1 \in [2^p]$ be the slot index for the p-th scheme. Sample a key pair

$$(\mathsf{pk}_p, \mathsf{sk}_p) \leftarrow \mathsf{sRABE.KeyGen}(\mathsf{crs}_p, i_p, f_p).$$

The final output is:

$$\mathsf{pk} = (\mathsf{ctr}, \mathsf{pk}_0, \ldots, \mathsf{pk}_k), \quad \mathsf{sk} = (\mathsf{ctr}, \mathsf{sk}_0, \ldots, \mathsf{sk}_k).$$

$\mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$: On input the common reference string $\mathsf{crs} = (\mathsf{crs}_0, \ldots, \mathsf{crs}_k)$, and the auxiliary data $\mathsf{aux} = (\mathcal{I}, \mathcal{PK}, N, \mathsf{mpk})$ where $\mathsf{mpk} = (\mathsf{ctr}_{\mathsf{aux}}, \mathsf{mpk}_0, \ldots, \mathsf{mpk}_k)$, a public key $\mathsf{pk} = (\mathsf{ctr}_{\mathsf{pk}}, \mathsf{pk}_0, \ldots, \mathsf{pk}_k)$, the registration algorithm proceeds as follows:

1. For each $p \in [0, k]$, let $i_p = (\mathsf{ctr}_{\mathsf{aux}} \bmod 2^p) + 1 \in [2^p]$ be the slot index for the p-th scheme.
2. For each $p \in [0, k]$, check that $\mathsf{sRABE.IsValid}(\mathsf{crs}_p, i_p, f_p, \mathsf{pk}_p) = 1$. In addition, check that $\mathsf{ctr}_{\mathsf{aux}} = \mathsf{ctr}_{\mathsf{pk}}$. If it fails, the algorithm halts and outputs the current aux and mpk.
3. For each $p \in [0, k]$, update $\mathcal{PK}[p, i_p] \leftarrow \mathsf{pk}$. If $i_p = 2^p$ (i.e., all slots in scheme p are filled), then:
   - Compute
   $$(\mathsf{mpk}'_p, \mathsf{hsk}'_{p,1}, \ldots, \mathsf{hsk}'_{p,2^p}) \leftarrow \mathsf{sRABE.Aggregate}(\mathsf{crs}_p, D_1[p, 1], \ldots, D_1[p, 2^p]).$$
   - For each $i \in [2^p]$, update $\mathcal{HSK}[\mathsf{ctr}_{\mathsf{aux}} + 1 - 2^p + i, p] \leftarrow \mathsf{hsk}'_{p,i}$.
   - If $i_p \neq 2^p$, set $\mathsf{mpk}'_p = \mathsf{mpk}_p$ (unchanged).
4. Define the updated master public key $\mathsf{mpk}' = (\mathsf{ctr}_{\mathsf{aux}} + 1, \mathsf{mpk}'_0, \ldots, \mathsf{mpk}'_k)$.
5. Finally, output the updated master public key and auxiliary data:
$$(\mathsf{mpk}', \mathsf{aux}') = (\mathsf{mpk}', (\mathsf{ctr}_{\mathsf{aux}} + 1, \mathcal{PK}, \mathcal{HSK}, \mathsf{mpk}')).$$

$\mathsf{Encrypt}(\mathsf{mpk}, \mathbf{w}, \mu)$: On input the master public key $\mathsf{mpk} = (\mathsf{ctr}, \mathsf{mpk}_0, \ldots, \mathsf{mpk}_\ell)$, an attribute $\mathbf{w} \in \mathcal{X}$, and a message $\mu \in \mathcal{M}$, the encryption algorithm proceeds as follows:

1. For each $p \in [0, k]$, compute
$$\mathsf{ct}_p \leftarrow \begin{cases} \mathsf{sRABE.Encrypt}(\mathsf{mpk}_p, \mathbf{w}, \mu) & \text{if } \mathsf{mpk}_p \neq \perp, \\ \perp & \text{otherwise.} \end{cases}$$
2. Output the ciphertext $\mathsf{ct} = (\mathsf{ctr}, \mathsf{ct}_0, \ldots, \mathsf{ct}_k)$.

$\mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$: On input the common reference string $\mathsf{crs} = (\mathsf{crs}_0, \ldots, \mathsf{crs}_k)$, the auxiliary data $\mathsf{aux} = (\mathsf{ctr}_{\mathsf{aux}}, \mathcal{PK}, \mathcal{HSK}, \mathsf{mpk})$, and a public key $\mathsf{pk} = (\mathsf{ctr}_{\mathsf{pk}}, \mathsf{pk}_0, \ldots, \mathsf{pk}_k)$, the update algorithm proceeds as follows:

1. If $\mathsf{ctr}_{\mathsf{pk}} \geq \mathsf{ctr}_{\mathsf{aux}}$, output $\perp$. Otherwise, for each $p \in [0, k]$, set $\mathsf{hsk}_p \leftarrow \mathcal{HSK}[\mathsf{ctr}_{\mathsf{pk}} + 1, p]$.
2. Output the helper decryption key $\mathsf{hsk} = (\mathsf{hsk}_0, \ldots, \mathsf{hsk}_k)$.

$\mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, \mathsf{ct})$: On input a secret key $\mathsf{sk} = (\mathsf{ctr}_{\mathsf{sk}}, \mathsf{sk}_0, \ldots, \mathsf{sk}_k)$, a helper decryption key $\mathsf{hsk} = (\mathsf{hsk}_0, \ldots, \mathsf{hsk}_k)$, ciphertext $\mathsf{ct} = (\mathsf{ctr}_{\mathsf{ct}}, \mathsf{ct}_0, \ldots, \mathsf{ct}_k)$, the decryption algorithm proceeds as follows:

1. If $\mathsf{ctr}_{\mathsf{ct}} \leq \mathsf{ctr}_{\mathsf{sk}}$, output $\perp$. Otherwise, let $p \in [0, k]$ be the largest index such that the p-th bit of $\mathsf{ctr}_{\mathsf{ct}}$ and $\mathsf{ctr}_{\mathsf{sk}}$ differ (where bits are indexed from least significant to most significant).
2. If $\mathsf{hsk}_p = \perp$, output $\mathsf{GetUpdate}$. Otherwise, output the result $\mathsf{sRABE.Decrypt}(\mathsf{sk}_p, \mathsf{hsk}_p, \mathsf{ct}_p)$.

Correctness. Suppose the slotted scheme Π_{sRABE} is complete and perfectly correct, then the registered ABE scheme is correct. By synchronously managing the counters and auxiliary data, each state update and aggregation operation can accurately "capture" the user registration information, ensuring that decryption always relies on the correct system state.

Security. If Π_{sRBE} is a secure slotted registered ABE scheme, then the registered ABE scheme is secure. The details of the proof process can be referenced in [HLWW23]. Although our key-policy registered ABE requires the policy to be provided as an input in the key generation algorithm instead of being specified in the registration phase, we consider this a minor limitation because our scheme still preserves the user's right to choose their own keys without relying on any trusted key issuer.

5 Conclusion

Our work initiates a new black-box construction of key-policy registered ABE from evasive lattice assumptions in the standard model. An open question, however, is whether we can construct a registered ABE under the standard LWE assumption in the standard model. Furthermore, another challenge lies in reducing the size of the common reference string, which currently depends polynomially on the security parameter and the λ number of users N; achieving $\mathrm{poly}(\lambda)$-size under standard assumptions remains an open problem. Finally, existing registered ABE for circuits require fixed input lengths, limiting the applicability to dynamic data; constructing a concrete scheme supporting expressive predicates like Turing machines remains an open direction for flexible encryption design.

References

AAH+25. K. Asano, N. Attrapadung, K. Hara, K. Hashimoto, and Y. Watanabe. Key revocation in registered attribute-based encryption. Springer-Verlag, 2025.

ABB10. S. Agrawal, D. Boneh, and X. Boyen. Efficient lattice (h) ibe in the standard model. In *Advances in Cryptology–EUROCRYPT 2010: 29th Annual International Conference on the Theory and Applications of Cryptographic Techniques, French Riviera, May 30–June 3, 2010. Proceedings 29*, pages 553–572. Springer, 2010.

Ajt96. M. Ajtai. Generating hard instances of lattice problems. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 99–108, 1996.

ARYY23. S. Agrawal, M. Rossi, A. Yadav, and S. Yamada. Constant input attribute based (and predicate) encryption from evasive and tensor lwe. In *Annual International Cryptology Conference*, pages 532–564. Springer, 2023.

AT24. N. Attrapadung and J. Tomida. A modular approach to registered abe for unbounded predicates. In *Annual International Cryptology Conference*, pages 280–316. Springer, 2024.

BDJ+24. P. Branco, N. Döttling, A. Jain, G. Malavolta, S. Mathialagan, S. Peters, and V. Vaikuntanathan. Pseudorandom obfuscation and applications. *Cryptology ePrint Archive*, 2024.

BGG+14. D. Boneh, C. Gentry, S. Gorbunov, S. Halevi, V. Nikolaenko, G. Segev, V. Vaikuntanathan, and D. Vinayagamurthy. Fully key-homomorphic encryption, arithmetic circuit abe and compact garbled circuits. In *Advances in Cryptology–EUROCRYPT 2014: 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings 33*, pages 533–556. Springer, 2014.

BÜW24. C. Brzuska, A. Ünal, and I. K. Woo. Evasive lwe assumptions: Definitions, classes, and counterexamples. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 418–449. Springer, 2024.

CES21. K. Cong, K. Eldefrawy, and N. P. Smart. Optimizing registration based encryption. In *IMA International Conference on Cryptography and Coding*, pages 129–157. Springer, 2021.

CHW25. J. Champion, Y.-C. Hsieh, and D. J. Wu. Registered abe and adaptively-secure broadcast encryption from succinct lwe. *Cryptology ePrint Archive*, 2025.

DJM+25. N. Döttling, A. Jain, G. Malavolta, S. Mathialagan, and V. Vaikuntanathan. Simple and general counterexamples for private-coin evasive lwe. *Cryptology ePrint Archive*, 2025.

DKL+23. N. Döttling, D. Kolonelos, R. W. Lai, C. Lin, G. Malavolta, and A. Rahimi. Efficient laconic cryptography from learning with errors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 417–446. Springer, 2023.

DKW21. P. Datta, I. Komargodski, and B. Waters. Decentralized multi-authority abe for dnf s from lwe. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 177–209. Springer, 2021.

DMQ25. J. Dujmovic, G. Malavolta, and W. Qi. Registration-based encryption in the plain model. *Cryptology ePrint Archive*, 2025.

FKP23. D. Fiore, D. Kolonelos, and P. d. Perthuis. Cuckoo commitments: registration-based encryption and key-value map commitments for large spaces. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 166–200. Springer, 2023.

FWW23. C. Freitag, B. Waters, and D. J. Wu. How to use (plain) witness encryption: registered abe, flexible broadcast, and more. In *Annual International Cryptology Conference*, pages 498–531. Springer, 2023.

GGH+16. S. Garg, C. Gentry, S. Halevi, M. Raykova, A. Sahai, and B. Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. *SIAM Journal on Computing*, 45(3):882–929, 2016.

GHM+19. S. Garg, M. Hajiabadi, M. Mahmoody, A. Rahimi, and S. Sekar. Registration-based encryption from standard assumptions. In *IACR international workshop on public key cryptography*, pages 63–93. Springer, 2019.

GHMR18. S. Garg, M. Hajiabadi, M. Mahmoody, and A. Rahimi. Registration-based encryption: removing private-key generator from ibe. In *Theory of Cryptography: 16th International Conference, TCC 2018, Panaji, India, November 11–14, 2018, Proceedings, Part I 16*, pages 689–718. Springer, 2018.

GKMR23. N. Glaeser, D. Kolonelos, G. Malavolta, and A. Rahimi. Efficient registration-based encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1065–1079, 2023.

GLWW24. R. Garg, G. Lu, B. Waters, and D. J. Wu. Reducing the crs size in registered abe systems. In *Annual International Cryptology Conference*, pages 143–177. Springer, 2024.

GPSW06. V. Goyal, O. Pandey, A. Sahai, and B. Waters. Attribute-based encryption for fine-grained access control of encrypted data. In *Proceedings of the 13th ACM conference on Computer and communications security*, pages 89–98, 2006.

GPV08. C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 197–206, 2008.

GV20. R. Goyal and S. Vusirikala. Verifiable registration-based encryption. In *Advances in Cryptology–CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part I 40*, pages 621–651. Springer, 2020.

HLWW23. S. Hohenberger, G. Lu, B. Waters, and D. J. Wu. Registered attribute-based encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 511–542. Springer, 2023.

KG10. A. Kate and I. Goldberg. Distributed private-key generators for identity-based cryptography. In *International Conference on Security and Cryptography for Networks*, pages 436–453. Springer, 2010.

LLNW16. B. Libert, S. Ling, K. Nguyen, and H. Wang. Zero-knowledge arguments for lattice-based accumulators: logarithmic-size ring signatures and group signatures without trapdoors. In *Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part II 35*, pages 1–31. Springer, 2016.

LW11. A. Lewko and B. Waters. Decentralizing attribute-based encryption. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 568–588. Springer, 2011.

LWW25. G. Lu, B. Waters, and D. J. Wu. Multi-authority registered attribute-based encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 3–33. Springer, 2025.

MP12. D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 700–718. Springer, 2012.

MPV24. S. Mathialagan, S. Peters, and V. Vaikuntanathan. Adaptively sound zero-knowledge snarks for up. In *Annual International Cryptology Conference*, pages 38–71. Springer, 2024.

Pei09. C. Peikert. Public-key cryptosystems from the worst-case shortest vector problem. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*, pages 333–342, 2009.

PS08. K. G. Paterson and S. Srinivasan. Security and anonymity of identity-based encryption with multiple trusted authorities. In *Pairing-Based Cryptography–Pairing 2008: Second International Conference, Egham, UK, September 1-3, 2008. Proceedings 2*, pages 354–375. Springer, 2008.

Reg09. O. Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009.

SW05. A. Sahai and B. Waters. Fuzzy identity-based encryption. In *Advances in Cryptology–EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005. Proceedings 24*, pages 457–473. Springer, 2005.

VWW22. V. Vaikuntanathan, H. Wee, and D. Wichs. Witness encryption and null-io from evasive lwe. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 195–221. Springer, 2022.

Wee22. H. Wee. Optimal broadcast encryption and cp-abe from evasive lattice assumptions. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 217–241. Springer, 2022.

WWW22. B. Waters, H. Wee, and D. J. Wu. Multi-authority abe from lattices without random oracles. In *Theory of Cryptography Conference*, pages 651–679. Springer, 2022.

Yao82. A. C. Yao. Theory and application of trapdoor functions. In *23rd Annual Symposium on Foundations of Computer Science (SFCS 1982)*, pages 80–91. IEEE, 1982.

ZZC+25. Z. Zhu, K. Zhang, Z. Chen, J. Gong, and H. Qian. Black-box registered abe from lattices. *Cryptology ePrint Archive*, 2025.

ZZGQ23. Z. Zhu, K. Zhang, J. Gong, and H. Qian. Registered abe via predicate encodings. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 66–97. Springer, 2023.

A Registered Attribute-Based Encryption

In this section, we present the formal definition of key-policy registered ABE, which is slightly different from the definition of ciphertext-policy registered ABE in [HLWW23].

Algorithms. Let λ be a security parameter and $\ell, d \in \mathbb{N}$. A registered key-policy attribute-based encryption scheme with attribute space $\mathcal{X}$ and policy space $\mathcal{P}$ consists of a tuple of efficient algorithms $\Pi_{\mathsf{RABE}} = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{RegPK}, \mathsf{Encrypt}, \mathsf{Update}, \mathsf{Decrypt})$:

$\mathsf{Setup}(1^\lambda, 1^d, 1^\ell)$: On input the security parameter λ, the circuit depth d and the input length ℓ, the setup algorithm outputs a common reference string crs.

$\mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, f) \to (\mathsf{pk}_i, \mathsf{sk}_i)$: On input the common reference string crs, auxiliary state aux, and a policy $f \in \mathcal{P}_{d,\ell}$, the key generation algorithm outputs a public key pk and a secret key sk.

$\mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}) \to (\mathsf{mpk}, \mathsf{aux}')$: On input the common reference string crs, auxiliary state aux, and a public key pk, the registration algorithm outputs the master public key mpk and an updated state aux'. This algorithm is deterministic.

$\mathsf{Encrypt}(\mathsf{mpk}, \mathbf{w}, \mu) \to \mathsf{ct}$: On input the master public key mpk, an attribute $\mathbf{w} \in \mathcal{X}_\ell$, and a message $\mu \in \{0,1\}$, the encryption algorithm outputs a ciphertext ct.

$\mathsf{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}) \to \mathsf{hsk}$: On input the common reference string crs, auxiliary state aux, and a public key, the update algorithm outputs a helper decryption key hsk. This algorithm is deterministic.

$\mathsf{Decrypt}(\mathsf{sk}, \mathsf{hsk}, \mathbf{w}, \mathsf{ct}) \to \mu$: On input the master public key mpk, a secret key sk, a helper decryption key hsk, an attribute $\mathbf{w}$, and a ciphertext ct, the decryption algorithm either outputs a message $\mu \in \{0,1\}$ or a special flag $\mathsf{GetUpdate}$ to indicate that the current helper decryption key needs to be updated. This algorithm is deterministic.

Definition A1 (Bounded Registered ABE). *If there is an a priori bound on the number of registered users in the system, we refer to it as a bounded registered ABE scheme* Π_{RABE}. *In this setting, the Setup algorithm takes an upper bound parameter* 1^N *as input, which specifies the maximum number of registered users supported by the scheme. In the correctness and security definitions, the adversary specifies the upper bound* 1^N *at the start. Moreover, in the game, the adversary is allowed to make at most* N *registration queries.*

Correctness. A registered ABE scheme satisfies the correctness requirement, which means that as long as all algorithms are executed according to the protocol, in every valid interaction and for any properly generated ciphertext, the system's decryption algorithm will recover the originally encrypted message, or, if necessary, prompt the user to update the helper decryption key. For a security parameter λ and an adversary $\mathcal{A}$, we define the correctness experiment as follows:

- **Setup Phase:** Given the security parameter λ, the circuit depth d, and the input length ℓ, the challenger samples a common reference string $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell, 1^d)$ and provides it to $\mathcal{A}$. Next, the challenger initializes the internal state by setting $\mathsf{aux} = \perp$ and $\mathsf{mpk} = \perp$, and also initializes two counters: $\mathrm{ctr}[\mathrm{reg}] = 0$ to record the number of registration queries, and $\mathrm{ctr}[\mathrm{enc}] = 0$ to record the number of encryption queries. Finally, the challenger initializes the target key's registration index by setting $\mathrm{ctr}[\mathrm{reg}]^* = \infty$.
- **Query phase:** During the query phase, the adversary $\mathcal{A}$ is able to make the following queries:
  - **Register non-target key query:** In this query, the adversary $\mathcal{A}$ submits a public key pk along with a policy $f \in \mathcal{P}$. The challenger increments the registration counter $\mathrm{ctr}[\mathrm{reg}] \leftarrow \mathrm{ctr}[\mathrm{reg}] + 1$ and registers the key by computing: $(\mathsf{mpk}_{\mathrm{ctr}[\mathrm{reg}]}, \mathsf{aux}') \leftarrow \mathrm{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$. The challenger updates its auxiliary state $\mathsf{aux} \leftarrow \mathsf{aux}'$ and responds to $\mathcal{A}$ with the tuple $(\mathrm{ctr}[\mathrm{reg}], \mathsf{mpk}_{\mathrm{ctr}[\mathrm{reg}]}, \mathsf{aux})$.
  - **Register target key query:** In this query, the adversary $\mathcal{A}$ specifies a target decryption policy $f^* \in \mathcal{P}$. If the target registration index is already set, i.e., $\mathrm{ctr}[\mathrm{reg}]^* \neq \infty$, then the challenger replies with $\perp$. Otherwise, the challenger increments the registration counter $\mathrm{ctr}[\mathrm{reg}] \leftarrow \mathrm{ctr}[\mathrm{reg}] + 1$, generates a key pair $(\mathsf{pk}^*, \mathsf{sk}^*) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, f^*)$ and registers the key by computing $(\mathsf{mpk}_{\mathrm{ctr}[\mathrm{reg}]}, \mathsf{aux}') \leftarrow \mathrm{Register}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*)$. It then computes the helper key $\mathsf{hsk}^* \leftarrow \mathrm{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*)$. The challenger updates its auxiliary state $\mathsf{aux} \leftarrow \mathsf{aux}'$,

stores the target registration index $\mathrm{ctr}[\mathrm{reg}]^* \leftarrow \mathrm{ctr}[\mathrm{reg}]$, and responds to $\mathcal{A}$ with the tuple $(\mathrm{ctr}[\mathrm{reg}], \mathsf{mpk}_{\mathrm{ctr}[\mathrm{reg}]}, \mathsf{aux}, \mathsf{pk}^*, \mathsf{hsk}^*, \mathsf{sk}^*)$.

- **Encryption query:** In this query, the adversary $\mathcal{A}$ submits an index i satisfying $\mathrm{ctr}[\mathrm{reg}]^* \leq i \leq \mathrm{ctr}[\mathrm{reg}]$, along with a message $\mu_{\mathrm{ctr}[\mathrm{enc}]} \in \mathcal{M}$ and an attribute $\mathbf{w}_{\mathrm{ctr}[\mathrm{enc}]} \in \mathcal{X}$. If a target key has not yet been registered, or if the target attribute $\mathbf{w}^*$ does not satisfy the policy $f_{\mathrm{ctr}[\mathrm{enc}]}$, then the challenger replies with $\perp$. Otherwise, it increments the encryption counter $\mathrm{ctr}[\mathrm{enc}] \leftarrow \mathrm{ctr}[\mathrm{enc}] + 1$ and computes the ciphertext $\mathsf{ct}_{\mathrm{ctr}[\mathrm{enc}]} \leftarrow \mathrm{Encrypt}(\mathsf{mpk}_i, \mathbf{w}_{\mathrm{ctr}[\mathrm{enc}]}, \mu_{\mathrm{ctr}[\mathrm{enc}]})$. The challenger then replies to $\mathcal{A}$ with $(\mathrm{ctr}[\mathrm{enc}], \mathsf{ct}_{\mathrm{ctr}[\mathrm{enc}]})$.
- **Decryption query:** In this query, the adversary $\mathcal{A}$ submits a ciphertext index $1 \leq j \leq \mathrm{ctr}[\mathrm{enc}]$. The challenger computes $\mu'_j \leftarrow \mathrm{Decrypt}(\mathsf{sk}^*, \mathsf{hsk}^*, \mathsf{ct}_j)$. If $\mu'_j = \mathsf{GetUpdate}$, the challenger updates the helper decryption key $\mathsf{hsk}^* \leftarrow \mathrm{Update}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}^*)$ and recomputes the decryption: $\mu'_j \leftarrow \mathrm{Decrypt}(\mathsf{sk}^*, \mathsf{hsk}^*, \mathsf{ct}_j)$. If the resulting message $\mu'_j \neq \mu_j$, the experiment halts and outputs $b = 1$.

If the adversary has finished making queries and the experiment has not halted due to a decryption error, then the experiment outputs $b = 0$.

We say that $\Pi_{\text{R-ABE}}$ is correct and efficient if, for all adversaries $\mathcal{A}$ making at most a polynomial number of queries, the following properties hold:

- **Correctness:** There exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, we have $\Pr[b = 1] = \mathsf{negl}(\lambda)$ in the correctness game.
- **Compactness:** Let N be the number of registration queries issued by the adversary in the above game. There exists a universal polynomial $\mathsf{poly}(\cdot)$ such that for all $i \in [N]$, $|\mathsf{mpk}_i| = \mathsf{poly}(\lambda, |\mathcal{X}|, \log i)$. We also require that the size of the helper decryption key satisfies $|\mathsf{hsk}^*| = \mathsf{poly}(\lambda, |\mathcal{X}|, \log N)$ at all points during the game.
- **Update efficiency:** Let N denote the number of registration queries made by the adversary in the game. Then, throughout the execution of the game, the challenger invokes the update algorithm Update at most $O(\log N)$ times, where each invocation runs in time $\mathsf{poly}(\log N)$ under the RAM model of computation. Specifically, we model Update as a RAM program with random access to its input, allowing its running time to be sublinear in the input size.

Security. For a security parameter λ and a challenge bit $b \in \{0, 1\}$, the security game between an adversary $\mathcal{A}$ and a challenger is defined as follows:

- **Setup phase:** On input the security parameter λ, the circuit depth d, and the input length ℓ, the challenger samples the common reference string $\mathsf{crs} \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell, 1^d)$ and sends crs to $\mathcal{A}$. It then initializes the auxiliary state $\mathsf{aux} = \perp$, a counter $\mathsf{ctr} = 0$ for the number of honest-key registration queries made by $\mathcal{A}$, an empty set for corrupted public keys $\mathcal{C} = \varnothing$, and an empty dictionary $\mathcal{D} = \varnothing$ mapping public keys to registered policy sets. For notational convenience, if $\mathsf{pk} \notin \mathcal{D}$, we define $\mathcal{D}[\mathsf{pk}] = \varnothing$.
- **Query phase:** The adversary $\mathcal{A}$ can now issue the following types of queries:
  - **Register honest key query:** The adversary specifies a policy $f \in \mathcal{P}$. The challenger increments the counter $\mathsf{ctr} \leftarrow \mathsf{ctr} + 1$ and samples a key pair $(\mathsf{pk}_{\mathsf{ctr}}, \mathsf{sk}_{\mathsf{ctr}}) \leftarrow \mathsf{KeyGen}(\mathsf{crs}, \mathsf{aux}, f)$ and registers $(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk}_{\mathsf{ctr}})$. The challenger updates its public key $\mathsf{mpk} \leftarrow \mathsf{mpk}'$, auxiliary state $\mathsf{aux} \leftarrow \mathsf{aux}'$, and dictionary $\mathcal{D}[\mathsf{pk}_{\mathsf{ctr}}] \leftarrow \mathcal{D}[\mathsf{pk}_{\mathsf{ctr}}] \cup \{f\}$. It replies to $\mathcal{A}$ with the tuple $(\mathsf{ctr}, \mathsf{mpk}', \mathsf{aux}', \mathsf{pk}_{\mathsf{ctr}})$.
  - **Corrupt honest key query:** The adversary specifies an index $1 \leq i \leq \mathsf{ctr}$. Let $(\mathsf{pk}_i, \mathsf{sk}_i)$ be the i-th key pair generated during an honest-key registration. The challenger adds pk_i to the corrupted set: $\mathcal{C} \leftarrow \mathcal{C} \cup \{\mathsf{pk}_i\}$, and responds to $\mathcal{A}$ with the secret key sk_i.
  - **Register malicious key query:** The adversary specifies a public key pk of policy $f \in \mathcal{P}$. The challenger registers the key by computing $(\mathsf{mpk}', \mathsf{aux}') \leftarrow \mathsf{RegPK}(\mathsf{crs}, \mathsf{aux}, \mathsf{pk})$. It updates its public key $\mathsf{mpk} \leftarrow \mathsf{mpk}'$, auxiliary state $\mathsf{aux} \leftarrow \mathsf{aux}'$, and adds pk to the corrupted set $\mathcal{C} \leftarrow \mathcal{C} \cup \{\mathsf{pk}\}$. The dictionary is also updated: $\mathcal{D}[\mathsf{pk}] \leftarrow \mathcal{D}[\mathsf{pk}] \cup \{f\}$. The challenger responds to $\mathcal{A}$ with $(\mathsf{mpk}', \mathsf{aux}')$.
- **Challenge phase:** The adversary specifies a challenge attribute $\mathbf{w}^* \in \mathcal{X}$, and the challenger responds with the challenge ciphertext $\mathsf{ct}^* \leftarrow \mathsf{Encrypt}(\mathsf{mpk}, \mathbf{w}^*, b)$.

– **Output phase:** At the end of the game, the adversary outputs a bit $b' \in \{0,1\}$.

Let $\mathcal{S} = \{f \in \mathcal{D}[\mathsf{pk}] : \mathsf{pk} \in \mathcal{C}\}$ be the set of policies associated with corrupted public keys. We say that an adversary $\mathcal{A}$ is admissible if for all $f \in \mathcal{S}$, it holds that $f(\mathbf{w}^*) = 1$. We say that a registered ABE scheme is secure if for all efficient and admissible adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, we have that $|\Pr[b' = 1 \mid b = 0] - \Pr[b' = 1 \mid b = 1]| = \mathsf{negl}(\lambda)$ in the security game described above.

Definition A2 (Selective Security) *Similar to Definition 2.13, we say that a key-policy registered ABE scheme satisfies selective security if the adversary is required to claim its attribute* $\mathbf{w}$ *at the beginning of the setup phase before seeing the common reference string.*

B Proof of Lemma 2.9

Lemma B1 *Let* $n, m, q, Q \in \mathbb{N}$ *be parameters, and let* $\chi_0, \chi, \gamma > 0$ *be Gaussian parameters satisfying* $m = \Omega(n \log q)$, $\gamma = \lambda^{\omega(1)}$, *and* $\chi = \gamma\chi_0\lambda^{\omega(1)}$. *Then, the hardness of the* $\mathsf{LWE}(n, m, q, \chi_0)$ *problem implies that:*

$$\left\{\mathbf{s}^\top(\mathbf{I}_n \otimes \mathbf{r}_i)\mathbf{b}_i + e_i^\top, \mathbf{b}_i, \mathbf{r}_i\right\}_{i\in[Q]} \approx_c \{c_i, \mathbf{b}_i, \mathbf{r}_i\}_{i\in[Q]}$$

where $\mathbf{b}_i \in \mathbb{Z}_q^n$, $\mathbf{s} \leftarrow \mathbb{Z}_q^{mn}$, $e_i \leftarrow D_{\mathbb{Z},\chi}$, $\mathbf{r}_i \leftarrow D_{\mathbb{Z}^m,\gamma}$ *and* $\mathbf{c}_i \leftarrow \mathbb{Z}_q$.

Proof. We prove the lemma via a sequence of games under the LWE assumption.

– $\mathcal{G}_0$: This is the real distribution given to the adversary:

$$\left\{\mathbf{s}^\top(\mathbf{I}_n \otimes \mathbf{r}_i)\mathbf{b}_i + e_i^\top, \ \mathbf{b}_i, \ \mathbf{r}_i\right\}_{i\in[Q]}.$$

– $\mathcal{G}_1$: We rewrite $\mathbf{s}^\top(\mathbf{I}_n \otimes \mathbf{r}_i)\mathbf{b}_i + e_i^\top$ using the tensor decomposition of $\mathbf{s}$, that is to say, We expand $\mathbf{s}$ into m blocks:

$$\mathbf{s} = \sum_{j=1}^{m} \mathbf{s}_j \otimes \boldsymbol{\epsilon}_j, \quad \text{where each } \mathbf{s}_j \in \mathbb{Z}_q^n.$$

Then we fix an index $1 \le i \le Q$ and rewrite the i-th sample:

$$\mathbf{s}^\top(\mathbf{I}_n \otimes \mathbf{r}_i)\mathbf{b}_i + e_i^\top = \sum_{j=1}^{m} \mathbf{r}_i[j] \cdot \langle \mathbf{s}_j, \mathbf{b}_i\rangle + e_i.$$

where $\mathbf{r}_i[j]$ is the j-th entry of $\mathbf{r}_i$. This is a syntactic change. Thus, $\mathcal{G}_1 \equiv \mathcal{G}_0$.

– $\mathcal{G}_2$: We now add auxiliary noise to introduce an LWE instance. For each $i \in [Q]$, $j \in [m]$, sample $e'_{i,j} \leftarrow D_{\mathbb{Z},\chi_0}$, the adversary is given:

$$\left\{\sum_{j=1}^{m} \mathbf{r}_i[j] \cdot (\langle \mathbf{s}_j, \mathbf{b}_i\rangle + e'_{i,j}) + e_i, \ \mathbf{b}_i, \ \mathbf{r}_i\right\}_{i\in[Q]}.$$

The only distinction between this game and the previous one lies in the noise term. In this game, the total noise becomes $e_i + \sum_{j=1}^{m} r_i[j] \cdot e'_{i,j}$. By Lemma 2.1, since $\chi = \lambda^{\omega(1)} \cdot \gamma\chi_0$, this transformation is statistically indistinguishable. Thus, $\mathcal{G}_2 \approx_s \mathcal{G}_1$.

– $\mathcal{G}_3$: In this game, we replace each $(\langle \mathbf{s}_j, \mathbf{b}_i\rangle + e'_{i,j})$ with a uniform random $c'_{j,i} \leftarrow \mathbb{Z}_q$. The adversary is given:

$$\left\{\sum_{j=1}^{m} r_i[j] \cdot c'_{j,i} + e_i, \ \mathbf{b}_i, \ \mathbf{r}_i\right\}_{i\in[Q]}.$$

Under the LWE assumption (on distinguishing $\langle \mathbf{s}_j, \mathbf{b}_i\rangle + e'_{i,j}$ from uniform), this is computationally indistinguishable: $\mathcal{G}_3 \approx_c \mathcal{G}_2$.

– $\mathcal{G}_4$: In this game, we define $\mathbf{c}'_i := (c'_{1,i}, \ldots, c'_{m,i}) \in \mathbb{Z}_q^m$. Then:

$$z_i = \langle \mathbf{r}_i, \mathbf{c}'_i\rangle + e_i.$$

Now we replace z_i with uniform $c_i \leftarrow \mathbb{Z}_q$. This is computationally indistinguishable under the LWE assumption. Thus, $\mathcal{G}_4 \approx_c \mathcal{G}_3$.